

SEMESTER :3
CST 205: OBJECT ORIENTED PROGRAMMING USING JAVA
MODEL QUESTION SET – I
PART A

Answer all Questions. Each question carries 3 Marks.

1. Briefly explain the portable, secure and robust features of Java?

Answer

1. Portable

- Java is portable because it facilitates to carry the Java bytecode to any platform.
- It doesn't require any implementation.

2. Secured

- Java is best known for its security.
- Java is secured because:
 - **No explicit pointer**
 - **Java Programs run inside a virtual machine sandbox.**
 - **Class loader:** Class loader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
 - **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access right to objects.

3. Robust

- Robust simply means strong. Java is robust because:
 - It uses strong memory management.
 - There is a lack of pointers that avoids security problems.
 - There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
 - There are exception handling and the type checking mechanism in Java. All these points make Java robust.
-

2. Describe the concepts of object and class with a suitable Java program?

Answer

- Class is the core of Java language.
- It can be defined as a template that describe the behaviors and states of a particular entity.
- A class defines new data type. Once defined this new type can be used to create object of that type.
- Object is an instance of class.
- A class contain both data and methods that operate on that data.
- The data or variables defined within a class are called **instance variables** and the code that operates on this data is known as **methods**.
- Thus, the **instance variables** and **methods** are known as **class members**.

Java class Syntax

```

class class_name
{
    field;
    method;
}

```

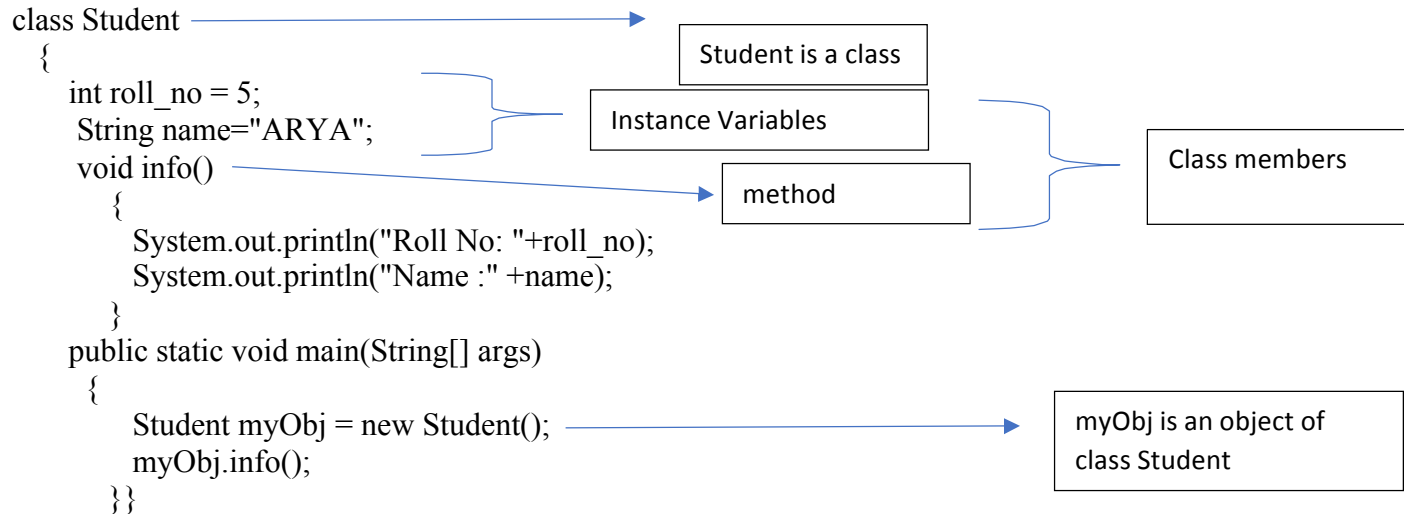
Java Object Syntax

```

className variable_name = new className();

```

JAVA PROGRAM



OUTPUT

```

Roll No: 5
Name : ARYA

```

3. Explain the concept of method overriding with an example?

Answer

- If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.
- Implementation within the sub class overrides (replaces) the inherited implementation from the parent class.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass (Parent class).
- Method overriding is used for runtime polymorphism.

Rules for Java Method Overriding

- The method must have the same name as in the parent class.
- The method must have the same parameter as in the parent class.
- There must be an IS-A relationship (inheritance).

Java Program to illustrate the use of Java Method Overriding

```

class Vehicle
{
    void run()

```

OUTPUT

```

Bike is running safely.

```

```

{
System.out.println("Vehicle is running");
}
}
class Bike2 extends Vehicle
{
void run()
{
System.out.println("Bike is running safely.");
}
}
class Demo
{
public static void main(String args[])
{
Bike2 obj = new Bike2();
obj.run(); } }

```

4. What is the use of the keyword final in Java?

Answer

- The final keyword in java is used to restrict the user.

• **Final Variable**  **To create constant variables**

If a variable is declared as final, they cannot change the value of final variable (It will be constant).

- Syntax: **final variable_name;**

Java program using Final keyword

```

class A
{
    final int a=10;
}
class Demo1
{
    public static void main (String args[])
    {
        A obj =new A ();
        System.out.println(obj.a=30); } }

```

OUTPUT

Demo1.java:11: error: cannot assign a value to final variable a

System.out.println(obj.a=30);

^

1 error

5. Explain the concept of streams?

Answer

STREAM

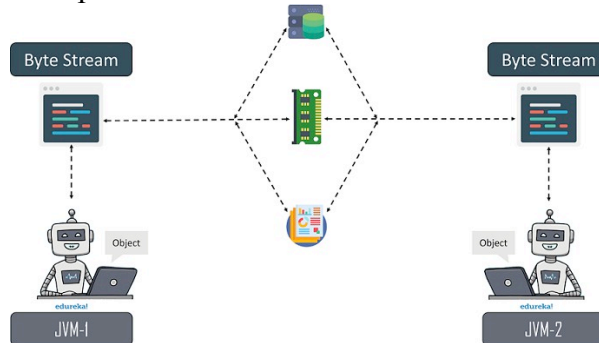
- A stream is a sequence of data.
- In Java, a stream is composed of bytes.
- Java uses the concept of a **stream** to make I/O operation fast.
 - It's called a stream because it is like a stream of water that continues to flow.
- In Java, 3 streams are created for us automatically.
- All these streams are attached with the console.
 - System.out:** standard output stream

- `System.out` normally outputs the data that writes to it to the console / terminal.
- **Example:** `System.out.println("simple message");`
- 2. **System.in:** standard input stream
 - which is typically connected to keyboard input of console programs.
 - **Example:** `int i=System.in.read();` //returns ASCII code of 1st character
- 3. **System.err:** standard error stream
 - works like `System.out` except it is normally only used to output error texts.
 - **Example:** `System.err.println("error message");`

6. Explain any two applications of Serialization?

Answer

- **Serialization** in Java is the process of converting the Java code *Object* into a *Byte Stream*, to transfer the Object Code from one Java Virtual machine to another and recreate it using the process of *Deserialization*.



Why do we need Serialization in Java or Applications of Serialization?

- **Communication:** Serialization involves the procedure of object *serialization* and *transmission*. This enables multiple computer systems to design, share and execute objects simultaneously.
- **Caching:** The time consumed in building an object is more compared to the time required for de-serializing it. Serialization minimizes time consumption by *caching* the giant objects.
- **Deep Copy:** *Cloning* process is made simple by using Serialization. An exact *replica* of an object is obtained by serializing the object to a *byte array*, and then de-serializing it.
- **Cross JVM Synchronization:** The major advantage of Serialization is that it works across different JVMs that might be running on different *architectures* or *Operating Systems*.
- **Persistence:** The State of any object can be directly stored by applying Serialization on to it and stored in a *database* so that it can be *retrieved later*

7. Distinguish the usage of “==” and equals() method when comparing String type?

==	equals()
It is a operator	It is a method
Used for for reference comparison (address comparison)	Used for content comparison .
== checks if both objects point to the same memory location	evaluates to the comparison of values in the objects.

// Java program to understand the concept of == operator and equals()

```
public class Test {
    public static void main(String[] args)
```

Output:

false

true

```

{
    String s1 = new String("HELLO");
    String s2 = new String("HELLO");
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));
}
}

```

Explanation:

- Here we are creating two objects namely s1 and s2.
- Both s1 and s2 refers to different objects.
- When we use == operator for s1 and s2 comparison then the result is false as both have different addresses in memory.
- Using equals, the result is true because its only comparing the values given in s1 and s2

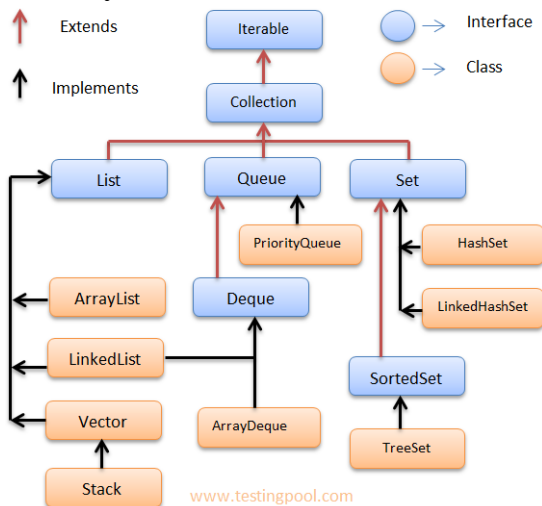
8. What are Collections in Java? Explain any one Collection interface in Java?

Answer

Collection Framework IN JAVA

- Java Collection means a single unit of objects.
- The java collections framework is a collection of interfaces and classes which help the programmer to perform operations on data like sorting, searching, deleting, manipulating etc.
- Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).
- The java.util package contains all the classes and interfaces for the Collection framework.

Hierarchy of Collection Framework:



List Interfaces

- In Java, the List interface is an ordered collection that allows us to store and access elements sequentially.
- It extends the Collection interface.
- Classes that Implement List interfaces are:
 - ArrayList
 - LinkedList
 - Vector
 - Stack

Java ArrayList Class

- The ArrayList class of the Java collections framework provides the functionality of resizable-arrays.
- It implements the List interface.

- It allows us to create resizable arrays.

The ArrayList class provides various methods to perform different operations on arrays.

- | | |
|--|--------------------|
| ▪ add (): To add a single element to the arraylist | 1. Add elements: |
| ▪ get (): to access an element from the arraylist | 2. Access elements |
| ▪ set (): To change element of the arraylist | 3. Change elements |
| ▪ remove (): to remove an element from the arraylist | 4. Remove elements |

9. Explain any two properties of Swing components in Java?

ANSWER

Features of Swing

1. Swing Components Are Lightweight

- This means that they are written entirely in Java and do not map directly to platform-specific peers.

2. Swing Supports a Pluggable Look and Feel

- SWING based GUI Application look and feel can be changed at run-time, based on available values.

3. Swing uses MVC Architecture

- Java's Swing components have been implemented using the model-view controller (MVC) model.
- Any Swing component can be viewed in terms of three independent aspects:
 - what state it is in (its model),
 - how it looks (its view), and
 - what it does (its controller).

10. Explain JLabel component. With suitable examples explain any two of its constructors.

Answer

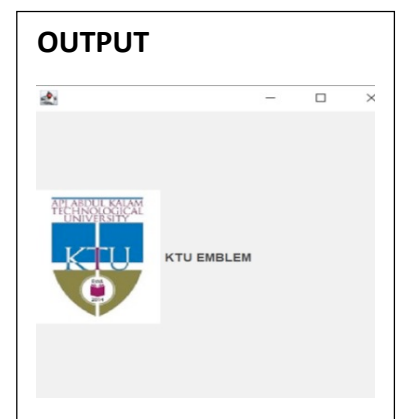
JLabel Constructors used are:

- JLabel(Icon ic)
- JLabel(String str)
- JLabel(String str,Icon ic,int align)
 - Here Icon is abstract class that cannot be instantiated.
 - ImageIcon is a class that extends Icon.
 - So to load images the following statement can be used:
ImageIcon ic=new ImageIcon("filename");
 where filename is a string quantity.

JLabel Example

```
import javax.swing.*;
public class SimpleLabel extends JFrame
{
    SimpleLabel()
    {
```

```
        ImageIcon ic=new ImageIcon("C:\\Users\\kiran\\OneDrive\\Pictures\\Desktop\\KTU.jpg");
        JLabel jl=new JLabel("KTU EMBLEM",ic,JLabel.LEFT);
        setSize(350,350);
        setVisible(true);
    }
```



```

add(jl);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String args[])
    {
        new SimpleLabel();
    }
}

```

Part B

Answer any one question completely from each module.

11. (a) Describe in detail any three Object Oriented Programming principles. Illustrate with suitable examples. (9)

Answer:

OBJECTS

- ✓ Objects are simply instants of a class.
- ✓ Objects are real world entities which has existence. Eg: Person, Place, bank account etc
- ✓ Objects are the basic run time entities in an object-oriented system.
- ✓ Each object should have a unique identity.

CLASSES

- ✓ Group of objects with similar features are grouped into class
- ✓ Class is a way to bind data and functions into a single unit.
- ✓ Each class should have a class name.
- ✓ Example: Mango, Apple, Grapes are objects of class Fruit.

DATA ABSTRACTION

- ✓ Providing only the essential features without providing the background details.
- ✓ Since classes using data abstraction, it is also called as Abstract Data Types (ADT).

ENCAPSULATION

- ✓ The wrapping up of data and functions into a single unit is known as encapsulation.
- ✓ The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it.
- ✓ The insulation of data from direct access by the program is called data hiding or information hiding.

11 (b) What is Java Runtime Environment? What is the role of Java Virtual Machine in it? (5)

OR

Explain JVM Architecture?

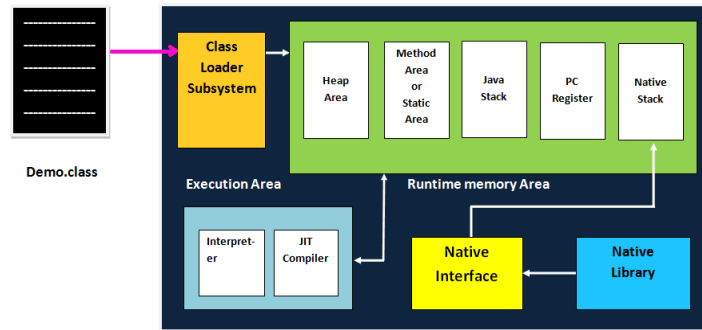
Answer:

JAVA RUNTIME ENVIRONMENT

- JRE stands for “Java Runtime Environment” and may also be written as “Java RTE.”
- The Java Runtime Environment provides the minimum requirements for executing a Java application:
 - it consists of the Java Virtual Machine (JVM), core classes, and supporting files.
- The JRE is the software environment in which programs compiled for a typical JVM implementation can run.
- The runtime system includes:
 - Code necessary to run Java programs, dynamically link native methods, manage memory, and handle exceptions.
 - Implementation of the JVM

Role of Java Virtual Machine in JRE

- **JVM (Java Virtual Machine)** is a software.
- It is a specification that provides Runtime environment in which java bytecode can be executed.



Operation of JVM

JVM mainly performs following operations.

- Allocating sufficient memory space for the class properties.
- Provides runtime environment in which java bytecode can be executed
- Converting byte code instruction into machine level instruction.

JVM is separately available for every Operating System while installing java software so that **JVM is platform dependent**.

Class loader subsystem:

Class loader subsystem will load the **.class** file into java stack and later sufficient memory will be allocated for all the properties of the java program into following five memory locations.

1. **Heap area:** In which object references will be stored.
2. **Method area:** In which static variables non-static and static method will be stored.
3. **Java stack:** In which all the non-static variable of class will be stored and whose address referred by object reference.
4. **PC register:** Which holds the address of next executable instruction that means that use the priority for the method in the execution process.
5. **Native stack:** Native stack holds the instruction of native code (other than java code) native stack depends on native library. Native interface will access interface between native stack and native library.

Execution Engine

- Which contains **Interpreter** and **JIT compiler** whenever any java program is executing at the first-time interpreter will comes into picture and it converts one by one byte code instruction into machine level instruction JIT compiler (just in time compiler) will comes into picture from the second time onward if the same java program is executing and it gives the machine level instruction to the process which are available in the buffer memory.

12. (a) Compare and contrast Java standard edition and Java enterprise edition. (5)

Answer:

	Java SE	Java EE
1	Java or Java SE provides basic functionality like defining basic types and objects.	Java EE provides APIs for running large-scale applications.
2	SE is a normal Java Specification.	EE is built upon JAVA SE. Provides functionalities like web applications, servlets etc

3	It consists of class libraries, virtual machines, deployment environment programming.	Java EE is a structured application with separate Client, Business, Enterprise layers.
4	Mostly used to develop APIs for Desktop Applications like antivirus software, game etc	Mainly used for web applications
5	Suitable for beginning Java developers	Suitable for experienced Java developers who build enterprise-wide applications.
6	Execution of Java SE based application is done using Java command from command prompt using main method class	Done using URL from browser software using Servlet.
7	To execute Java SE based Application just JDK/JRE software is sufficient.	To execute Java EE based application, must have JDK and server software.
8	From java SE based applications output should be printed on console using System.out.print() or System.out.println() method.	From Java EE based applications, must print output on browser using out.println() method.
9	Java SE has the following API's: 1. Applet 2. AWT 3. JDBC 5. Swing	Java EE has the following API's: 1. Servlet 2. Websocket 3. Java Faces

12)(b) Why is Java considered to be platform independent? What is the role of Bytecode in making Java platform independent? (9)

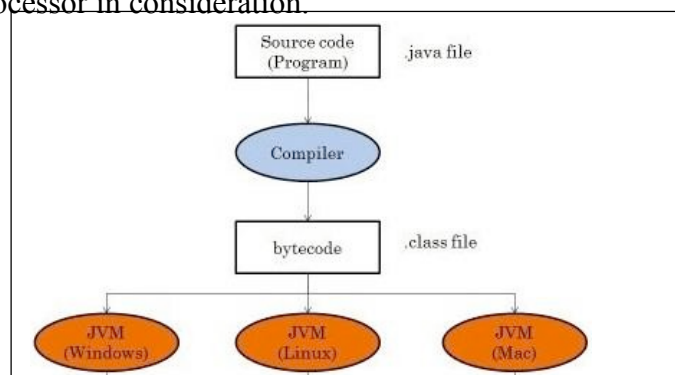
Answer:

Java Bytecode

- Bytecode is program code that has been compiled from source code into low-level code designed for a software interpreter.
- A popular example is Java bytecode, which is compiled from Java source code and can be run on a Java Virtual Machine (JVM).
- Java bytecode is the instruction set for the Java Virtual Machine.
- It acts similar to an assembler which is an alias representation of a C++ code.
- As soon as a java program is compiled, java bytecode is generated.
- In more apt terms, java bytecode is the machine code in the form of a .class file.
- With the help of java bytecode, we achieve platform independence in java.

How does it work?

- When write a program in Java, firstly, the compiler compiles that program, and a bytecode is generated for that piece of code.
- After the first compilation, the bytecode generated is now run by the Java Virtual Machine and not the processor in consideration.



- Resources required to run the bytecode are made available by the Java Virtual Machine, which calls the processor to allocate the required resources. JVM's are stack-based so they use stack implementation to read the codes.
- Java Virtual Machine (JVM) is an engine that provides runtime environment to drive the Java Code or applications.
- It converts Java bytecode into machine language. JVM is a part of Java Run Environment (JRE).
- In other programming languages, the compiler produces machine code for a particular system.
- However, Java compiler produces code for a Virtual Machine known as Java Virtual Machine.
 - Platform independent language means once compiled, can execute the program on any platform (OS).
 - Java is platform independent. Because the Java compiler converts the source code to bytecode, which is Intermediate Language. Bytecode can be executed on any platform (OS) using JVM (Java Virtual Machine).
 - Download JVM's (comes along with JDK or JRE) suitable to any operating system and, write a Java program, can run it on any system using JVM.

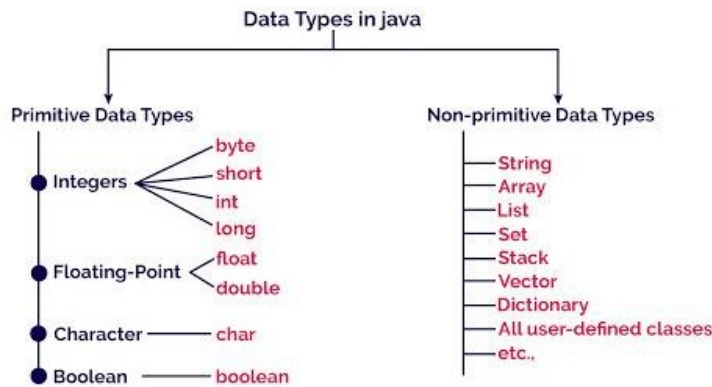
Advantage of Java Bytecode

- Platform independence is one of the main reasons for which James Gosling started the formation of Java and it is this implementation of bytecode which helps us to achieve this.
- Hence bytecode is a very important component of any Java program.
- The set of instructions for the JVM may differ from system to system but all can interpret the bytecode.
- Bytecodes are non-runnable codes and rely on the availability of an interpreter to execute and thus the JVM comes into play.

13. (a) Explain in detail the primitive data types in Java. (8)

Answer:

- Data type defines the values that a variable can take, for example if a variable has int data type, it can only take integer values.
- Data types specify the different sizes and values that can be stored in the variable.
- There are two types of data types in Java:
 - Primitive data types
 - Non-primitive data types



Primitive Data type

- **Primitive** types are the most basic **data types** available within the **Java** language.
- These data types are already hard coded into the compiler to be recognized when the program is executed.
- They are classified as
 1. Integers
 2. Floating -Point
 3. Character
 4. Boolean

1. Primitive Data type - Integer

- Java defines four integer types, and they are used for storing whole numbers.
 - a) byte (8 bit):** The **byte** data type can store whole numbers from -128(2^7) to 127($2^7 - 1$).
 - b) short (16 bit):** The **short** data type can store whole numbers from -32768(2^{15}) to 32767($2^{15} - 1$).
 - c) int (32 bit):** The **int** data type can store whole numbers from -2147483648 to 2147483647.
 - d) long (64 bit):** The **long** data type can store whole numbers from -9223372036854775808 to 9223372036854775807. This is used when int is not large enough to store the value. It should end the value with an "L".

PROGRAM

```

public class MyClass {
    public static void main(String[] args) {
        byte Num1 = -100;
        short Num2 = 32565;
        int Num3 = 123456789;
        long Num4 = 150000000000L;
        System.out.println(Num1);
        System.out.println(Num2);
        System.out.println(Num3);
        System.out.println(Num4);
    }
}
  
```

OUTPUT

```

-100
32565
123456789
150000000000
  
```

2. Primitive Data type – Floating point types

- are used for fractional numbers.
 - a) float:** The **float** data type can store fractional numbers. It should end the value with an "f".

- b) **double**: The **double** data type can store fractional numbers. It should end the value with a "d".

PROGRAM

```
public class MyClass {  
    public static void main(String[] args) {  
        float Num1 = 5.1234568f;  
        double Num2 = 19.99d;  
        System.out.println(Num1);  
        System.out.println(Num2);  
    }  
}
```

OUTPUT

```
5.123457  
19.99
```

3. **Primitive Data type – Characters**

- a. **char**: The **char** data type is used to store a **single** character.
The character must be surrounded by single quotes, like 'A' or 'c':

PROGRAM

```
public class MyClass {  
    public static void main(String[] args) {  
        char mygrade = 'O';  
        char a = 65, b = 66, c = 67;  
        System.out.println(mygrade);  
        System.out.println(a);  
        System.out.println(b);  
        System.out.println(c);  
    }  
}
```

OUTPUT

```
O  
A  
B  
C
```

4. **Primitive Data type – Booleans**

- a. **boolean**: A boolean data type is declared with the **boolean** keyword and can only take the values **true** or **false**.

PROGRAM

```
public class MyClass {  
    public static void main(String[] args) {  
        boolean isGood = true;  
        boolean isBad = false;  
        System.out.println(isGood);  
        System.out.println(isBad);  
    }  
}
```

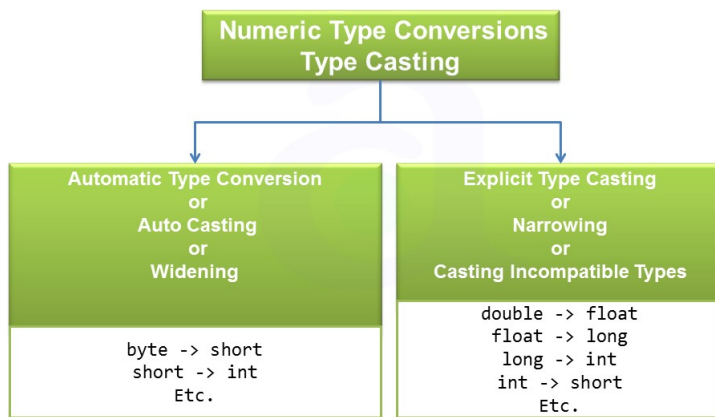
OUTPUT

```
true  
false
```

13)(b) Explain automatic type conversion in Java with an example. What are the two conditions required for it? (6)

Answer:

- When a data type is converted into another type is called type conversion(casting).
- When a variable is converted into a different type, the compiler basically treats the variable as of the new data type.
- **Type of Type Conversion in Java**
 1. Automatic Type Conversion/ Implicit Conversion/Widening
 2. Explicit Type Casting/Narrowing



Automatic Type Conversion/ Implicit Conversion/Widening

- converting a smaller type to a larger type size (called Type Conversion)
- Implicit or automatic conversion can happen if both types are compatible and target type is larger than source type.
- **byte -> short -> char -> int -> long -> float -> double.**

PROGRAM

```

public class type_conversion
{
    public static void main(String[] args) {
        int number = 25;
        float fval = number;
        System.out.println(fval);    }}
  
```

OUTPUT

25.0

- When one type of data is assigned to another type of variable, **an automatic type conversion will take place if the following two conditions are met:**
 - **The two types are compatible.**
 - **The destination type is larger than the source type.**
- When these two conditions are met, a widening conversion takes place.
 - For example, the int type is always large enough to hold all valid byte values, so no explicit cast statement is required.
- **char and boolean are not compatible with each other.**

14. a) Using a suitable Java program explain the difference between private and public members in the context of inheritance. (8 marks)

Answer:

- Inheritance is a mechanism of creating a new class from an existing class by inheriting the features of existing class and adding additional features of its own.
- When a class is derived from an existing class, all the members of the superclass are automatically inherited in the subclass.
- However, it is also possible to restrict access to fields and method of the superclass in the subclass.
- This is possible by applying the access Specifiers to the member of the superclass.
- If a subclass needs to access a superclass member, give that member private access.
- The private members of the superclass remain private (accessible within the superclass only) in the superclass and hence are not accessible directly to the members of the subclass.
- However, the subclass can access them indirectly through the inherited accessible methods of the superclass.

Using the private access specifier

class Baseclass

```

{
    private void m1()
    {
        System.out.println("Base class m1 method");
    }
}
class Derivedclass extends Baseclass
{

```

OUTPUT

```

MainClass.java:19: error: cannot find symbol
    d.m1();
      ^
symbol: method m1()
location: variable d of type Derivedclass
1 error

```

```

}
public class MainClass
{
    public static void main(String[] args)
    {
        Derivedclass d = new Derivedclass(); // Private members cannot be accessed due to not
        available in subclass.
        d.m1();
    }
}

```

Using the public access specifier

```

class Baseclass
{
    public void m1()
    {
        System.out.println("Base class m1 method");
    }
}
class Derivedclass extends Baseclass
{
}
public class MainClass
{
    public static void main(String[] args)
    {
        Derivedclass d = new Derivedclass(); // Private members cannot be accessed due to not
        available in subclass.
        d.m1();    } }

```

OUTPUT

```

Base class m1 method

```

14)b) Is it possible to use the keyword super within a static method? Give justification for your answer. (6 marks)

Answer:

- A static method or, block belongs to the class and these will be loaded into the memory along with the class.
- We can invoke static methods without creating an object. (using the class name as reference).
- Where, the "super" keyword in Java is used as a reference to the object of the super class.
- This implies that to use "super" the method should be invoked by an object, which static methods are not.
- Therefore, it is not possible to use "super" keyword from a static method.

```

class SuperClass {

```

```

protected String name;
}
public class SubClass extends SuperClass {
    private String name;
    public static void setName(String name) {
        super.name = name;
    }
    public void display() {
        System.out.println("name: "+super.name);
    }
    public static void main(String args[]) {
        new SubClass().display();
    }
}

```

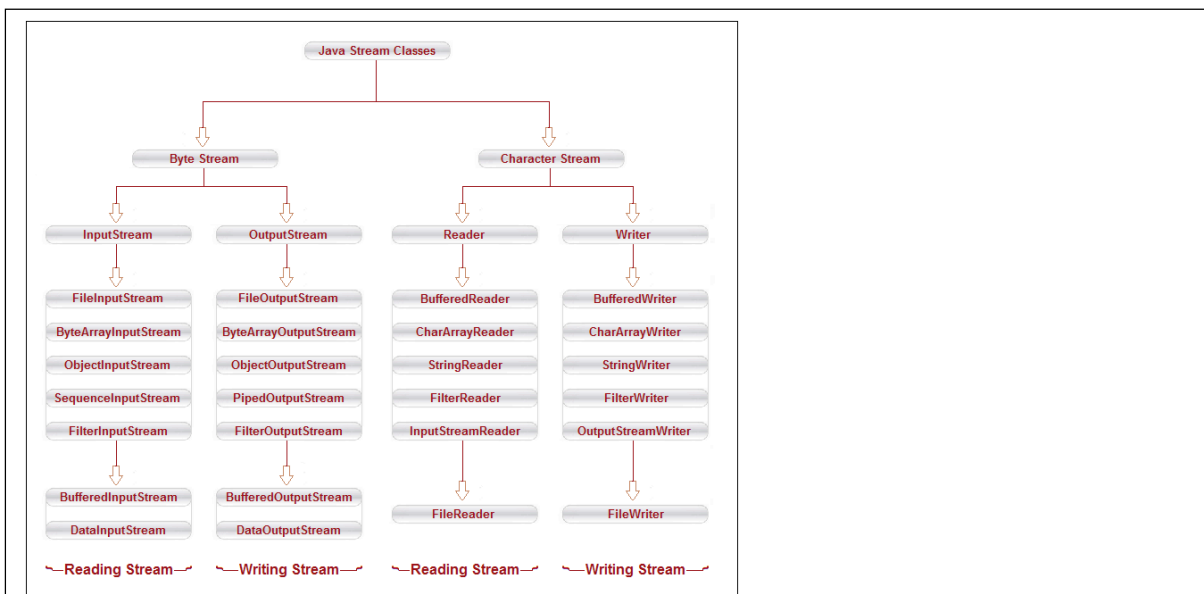
15) a) Explain in detail about byte streams and character streams with suitable code samples. (6 marks)

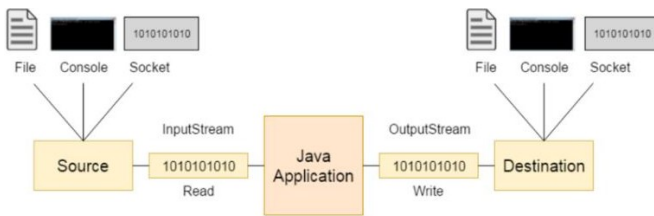
STREAM

- A stream is a sequence of data.
 - In Java, a stream is composed of bytes.
 - It's called a stream because it is like a stream of water that continues to flow.
 - In Java, 3 streams are created for us automatically.
 - All these streams are attached with the console.
4. **System.out**: standard output stream
 - System.out normally outputs the data that writes to it to the console / terminal.
 - **Example**: System.out.println("simple message");
 5. **System.in**: standard input stream
 - which is typically connected to keyboard input of console programs.
 - **Example**: int i=System.in.read(); //returns ASCII code of 1st character
 6. **System.err**: standard error stream
 - works like System.out except it is normally only used to output error texts.
 - **Example**: System.err.println("error message");

Types of Streams

- Depending upon the data a stream holds, it can be classified into:
 1. Byte Stream
 2. Character Stream





a) Java InputStream Class

- InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.

Useful methods of InputStream

1.	read()	reads the next byte of data from the input stream.
2.	available()	returns an estimate of the number of bytes that can be read from the current input stream.
3.	close()	is used to close the current input stream.

b) Java OutputStream class

- OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes.
- An output stream accepts output bytes and sends them to some sink.

Useful methods of OutputStream

1.	write(int)	is used to write a byte to the current output stream.
2.	write(byte[])	is used to write an array of byte to the current output stream.
3.	flush()	flushes the current output stream.
4.	close()	is used to close the current output stream.

Java program that read from a file and write to file by handling all file related exceptions using FileInputStream and FileOutputStream classes

PROGRAM

```
import java.io.*;
class Main
{
    public static void main(String a[]) throws IOException
    {
        try
        {
            FileInputStream f1= new FileInputStream("test.txt");
            FileOutputStream f2= new FileOutputStream("cp.txt");
            int c;
            while((c=f1.read())!=-1)
            {
                f2.write((char)c);
                System.out.print((char)c);
            }
            f1.close();
            f2.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println("File not found"); } } }
```

OUTPUT

India is my country.

test.txt

India is my country.

cp.txt

India is my country.

Character Stream

- Java Character streams are used to perform input and output for 16-bit unicode.
- Most frequently used classes are, **FileReader and FileWriter**.
- Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but the major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

Java program that read from a file and write to file by handling all file related exceptions using FileReader and FileWriter classes

```
import java.io.*;
class Main
{
    public static void main(String a[]) throws IOException
    {
        try
        {
            FileReader f1= new FileReader("test.txt");
            FileWriter f2= new FileWriter("cp.txt");
            int c;
            while((c=f1.read())!=-1)
            {
                f2.write((char)c);
                System.out.print((char)c);
            }
            f1.close();
            f2.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println("File not found");
        }
    }
}
```

OUTPUT

India is my country.

test.txt

India is my country.

cp.txt

India is my country.

15)b) Describe in detail about exception handling, try block and catch clause with the help of a suitable Java program.(8 marks)

What is Exception Handling?

- The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that normal flow of the application can be maintained.
- Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.
- The core advantage of exception handling is to maintain the normal flow of the application.
- An exception normally disrupts the normal flow of the application that is why we use exception handling.

Types of Java Exceptions

- There are mainly two types of exceptions:
 - checked and
 - unchecked.
- Here, an error is considered as the unchecked exception.
- According to Oracle, there are three types of exceptions:

1. **Checked Exception**
2. **Unchecked Exception**
3. **Error**

1. **Checked Exception**

- The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions.
- e.g. IOException, SQLException etc.
- Checked exceptions are **checked at compile-time**.

2. **Unchecked Exception**

- The classes which inherit RuntimeException are known as unchecked exceptions
- e.g. ArithmeticException, NullPointerException,
- Unchecked exceptions are not checked at compile-time, but they are **checked at runtime**.

3. **Error**

- Error is irrecoverable
- e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java try-catch block

- **The try statement allows you to define a block of code to be tested for errors while it is being executed.**
- **The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.**
- **The try and catch keywords come in pairs.**

Syntax of Java try-catch

```
try
{
    Block of code that may throw an exception
}
catch(Exception e)
{
    Block of code to handle errors
}
```

[Consider the following program: Problem without exception handling](#)

Example

```
public class TryCatchExample1 {
    public static void main(String[] args) {
        int data=50/0; //may throw exception
        System.out.println("rest of the code");
    }
}
```

OUTPUT

Exception in thread "main" java.lang.ArithmeticException: / by zero
at TryCatchExample1.main(TryCatchExample1.java:5)

[Solution by exception handling using try-catch block](#)

```
public class Main
{
    public static void main(String[] args)
    {
        try
        {
            int data=50/0; //may throw exception
        }
        // handling the exception
        catch(Exception e)
```

OUTPUT

Exception Occurred

REST OF THE CODE

```

{
    System.out.println("Exception Occurred");
}
System.out.println("REST OF THE CODE"); } }

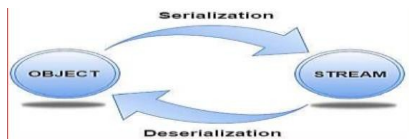
```

16.(a) Explain object streams in Java. Explain the role of Serializable interface with a suitable code sample. (8 marks)

- Serialization in Java is the process of converting the Java code Object into a Byte Stream, to transfer the Object Code from one Java Virtual machine to another and recreate it using the process of Deserialization.
- Most impressive is that the entire process is JVM independent, meaning an object can be serialized on one platform and deserialized on an entirely different platform.
- For serializing the object, we call the writeObject() method of ObjectOutputStream, and for deserialization we call the readObject() method of ObjectInputStream class.

Advantages of Java Serialization

- It is mainly used to travel object's state on the network (which is known as marshaling).



ObjectOutputStream class

- The ObjectOutputStream class is used to write primitive data types, and Java objects to an OutputStream.
- Only objects that support the java.io.Serializable interface can be written to streams.

Important Methods

Method	Description
writeObject(Object obj)	writes the specified object to the ObjectOutputStream.
flush()	flushes the current output stream.
close()	closes the current output stream.

ObjectInputStream class

- An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

Important Methods

Method	Description
readObject()	reads an object from the input stream.
close()	closes ObjectInputStream.

Example of Java Serialization

In this example, we are going to serialize the object of Student class. The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object. We are saving the state of the object in the file named f.txt.

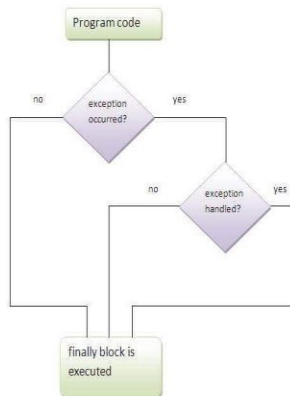
```
import java.io.*;
class Persist{
    public static void main(String args[]){
        try{
            //Creating the object
            Student s1 =new Student(211,"ravi");
            //Creating stream and writing the object
            FileOutputStream fout=new FileOutputStream("f.txt");
            ObjectOutputStream out=new ObjectOutputStream(fout);
            out.writeObject(s1);
            out.flush();
            //closing the stream
            out.close();
            System.out.println("success");
        }catch(Exception e){System.out.println(e);}
    }
}
```

OUTPUT
success

16 (b) Explain throw, throws and finally constructs with the help of a Java program.(6 marks)

Java finally block

- Java finally block is a block that is used to execute important code such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block.



Case 1: Java finally example where exception doesn't occur.

```
Public class CASE1
{
    public static void main(String args[])
    {
        try
        {
            int data=25/5;
            System.out.println(data);
        } catch(ArithmeticException e)
        {System.out.println("EXCEPTION");}
        finally {System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    }
}
```

OUTPUT

5
finally block is always executed
rest of the code...

Case 2

Java finally example where **exception occurs and not handled**.

```
public class CASE2{
    public static void main(String args[]){
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(NullPointerException e){System.out.println("HANDLE
        EXCEPTION");}
        finally{System.out.println("finally block is always executed");}
```

Output

Exception in thread "main" finally block is always executed

java.lang.ArithmeticException: / by zero at
TestFinallyBlock1.main(TestFinallyBlock1.java:4)

Case 3

Java finally example where exception occurs and handled.

```
public class CASE3{
    public static void main(String args[]){
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(ArithmeticException e)
        {
            System.out.println("HANDLED EXCEPTION");
        }
        finally{System.out.println("finally block is always
        executed");}
        System.out.println("rest of the code..."); }}
```

OUTPUT

HANDLED EXCEPTION

finally block is always executed

rest of the code...sss

Java throw keyword

- The Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or unchecked exception in java by throw keyword.
- The syntax of java throw keyword is given below.
throw exception;

```
public class MyClass {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Access denied - You must be
            at least 18 years old.");
        }
        else {
            System.out.println("Access granted - You are old enough!");
        }
    }

    public static void main(String[] args) {
        checkAge(15); // Set age to 15 (which is below 18...)
```

OUTPUT

Exception in thread "main"

java.lang.ArithmeticException: Access denied -
You must be at least 18 years old.

at MyClass.checkAge(MyClass.java:4)

at MyClass.main(MyClass.java:12)

Java throws keyword

- The Java throws keyword is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Syntax of java throws

```
return_type method_name() throws exception_class_name
{
    //method code
}
```

```
import java.io.*;
class M{
    void method()throws IOException{
        throw new IOException("device error");
    }
}

public class Testthrows4{
    public static void main(String args[])throws IOException{//declare exception
        M m=new M();
        m.method();

        System.out.println("normal flow...");
    }
}
```

OUTPUT

Exception in thread "main" java.io.IOException: device error at M.method(Testthrows4.java:4)
at Testthrows4.main(Testthrows4.java:10)

17. (a) Describe in detail the creation of a thread using the Runnable interface and the Thread class with suitable examples. (10 marks)

Creating a Thread

- Threads can be created by using two mechanisms:
 1. Extending the Thread class
 2. Implementing the Runnable Interface

Thread class:

- Thread class provide constructors and methods to create and perform operations on a thread.
- Commonly used **Constructors** of Thread class:
 - Thread ()
 - Thread (String name)
 - Thread (Runnable r)
 - Thread (Runnable r, String name)
- Thread class extends Object class and implements Runnable interface.
- Commonly used **methods** of Thread class:
 - public void run (): is used to perform action for a thread.
 - public void start (): starts the execution of the thread.
 - JVM calls the run () method on the thread

1. By extending the thread class

- Create a new class that extends Thread and create an instance of that class.

- The extending class must override the run() method -> entry point for the new thread
- It must also call start() ---> to begin execution of the new thread.

Steps

1. Use a run() method which is initiated by start() method

```
class A extends Thread
{
    public void run()
    {
        -----
    }
}
```

2. Invoking the run() method

- Creating the object of the corresponding thread class in the main() method class.

```
Thread_classname objName = new Thread_classname();
objName.start();    // invokes run() method
```

Java Thread Example by extending Thread class

```
class Multi extends Thread
{
    public void run()
    {
        System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        Multi t1=new Multi();
        t1.start();
    }
}
```

OUTPUT

thread is running...

2. Implementing the Runnable Interface

• Steps for using a Runnable interface in Java

- Create a class that implements the Runnable interface.
- override the run() method in the Runnable class.
- need to pass the Runnable object as a parameter to the constructor of the Thread class object while creating it.
- Then this Thread object is capable of executing our Runnable class.
- Finally, need to invoke the Thread object's start() method.

Java Thread Example by implementing Runnable interface

```
class Multi3 implements Runnable
{
    public void run ()
    {
        System.out.println("thread is running...");
    }
    public static void main (String args []) {
        Multi3 m1=new Multi3();
        Thread t1 =new Thread(m1);
        t1.start(); }}

```

Output:

thread is running...

17 (b) Explain List Interface. Mention any two exceptions thrown by its methods.

List Interfaces

- In Java, the List interface is an ordered collection that allows us to store and access elements sequentially.
- It extends the Collection interface.
- Classes that Implement List interfaces are:
 - ArrayList
 - LinkedList
 - Vector
 - Stack

Java ArrayList Class

- The ArrayList class of the Java collections framework provides the functionality of resizable-arrays.
- It implements the List interface.
- It allows us to create resizable arrays.
- Unlike arrays, arraylists can automatically adjust its capacity when we add or remove elements from it. Hence, arraylists are also known as dynamic arrays.

The ArrayList class provides various methods to perform different operations on arraylists.

- | | |
|--|--------------------|
| ▪ add (): To add a single element to the arraylist | 1. Add elements: |
| ▪ get (): to access an element from the arraylist | 2. Access elements |
| ▪ set (): To change element of the arraylist | 3. Change elements |
| ▪ remove (): to remove an element from the arraylist | 4. Remove elements |

Creating an ArrayList

```
ArrayList<Type> arrayList= new ArrayList<>();
```

Here, Type indicates the type of an arraylist.
import java.util.*;

```
class Main
{
    public static void main(String[] args)
    {
        // create ArrayList
        ArrayList<Integer> A = new ArrayList<>();

        // Add elements to ArrayList : add()
        A.add(0);
        A.add(2);
        A.add(6);
        A.add(8);
        A.add(10);
        System.out.println("EVEN ArrayList: " + A);
    }
}
```

OUTPUT

```
EVEN ArrayList: [0, 2, 6, 8, 10]
Updated EVEN ArrayList: [0, 2, 4, 6, 8, 10]
Element at index 3: 6
Updated EVEN ArrayList: [0, 2, 4, 12, 8, 10]
Updated EVEN ArrayList: [0, 2, 4, 8, 10]
```



```

    A.add(2,4);
    System.out.println("Updated EVEN ArrayList: " + A);
    int i=A.get(3);
    System.out.println("Element at index 3: " + i);
    A.set(3,12);
    System.out.println("Updated EVEN ArrayList: " + A);
    A.remove(3);
    System.out.println("Updated EVEN ArrayList: " + A);
}
}

```

Exceptions

1. ArrayIndexOutOfBoundsException

18. (a) Explain in detail the Delegation Event model for event handling in Java?

EVENT HANDLING

- Event Handling is the mechanism that controls the event and decides what should happen if an event occurs.
- This mechanism has the code which is known as **event handler** that is executed when an event occurs.
- Java Uses the **Delegation Event Model** to handle the events.
- This model defines the standard mechanism to generate and handle the events.

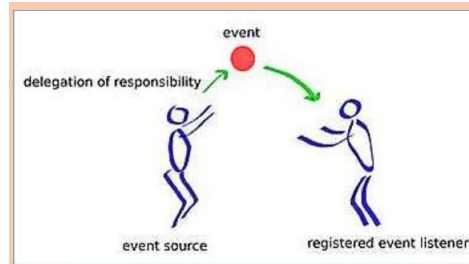
Explain Delegation Event Model?

- The Delegation Event Model has the following key participants namely
 - **Source: Generates Events**
 - The source is an object on which event occurs.
 - Source is responsible for providing information of the occurred event to it's handler.
 - Java provide as with classes for source object.
 - Some of the sources are:
 - **Button:** This class creates a labeled button.
 - **Check box:** a graphical component that can be in either an **on** (true) or **off** (false) state.
 - **List component:** presents the user with a scrolling list of text items.
 - **TextField** :a text component that allows for the editing of a single line of text.
 - **Choice control:** is used to show pop up menu of choices. Selected choice is shown on the top of the menu.
 - **Listener**
 - It is also known as event handler. Listener is responsible for generating response to an event.
 - From java implementation point of view the listener is also an object.
 - Listener waits until it receives an event. Once the event is received, the listener processes the event and then returns.

- The benefit of this approach is that the user interface logic is completely separated from the logic that generates the event.
- The user interface element is able to delegate the processing of an event to the separate piece of code.
- In this model, Listener needs to be registered with the source object so that the listener can receive the event notification.
- This is an efficient way of handling the event because the event notifications are sent only to those listeners that want to receive them.

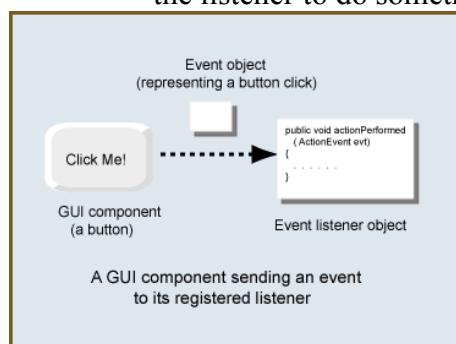
How Events are handled

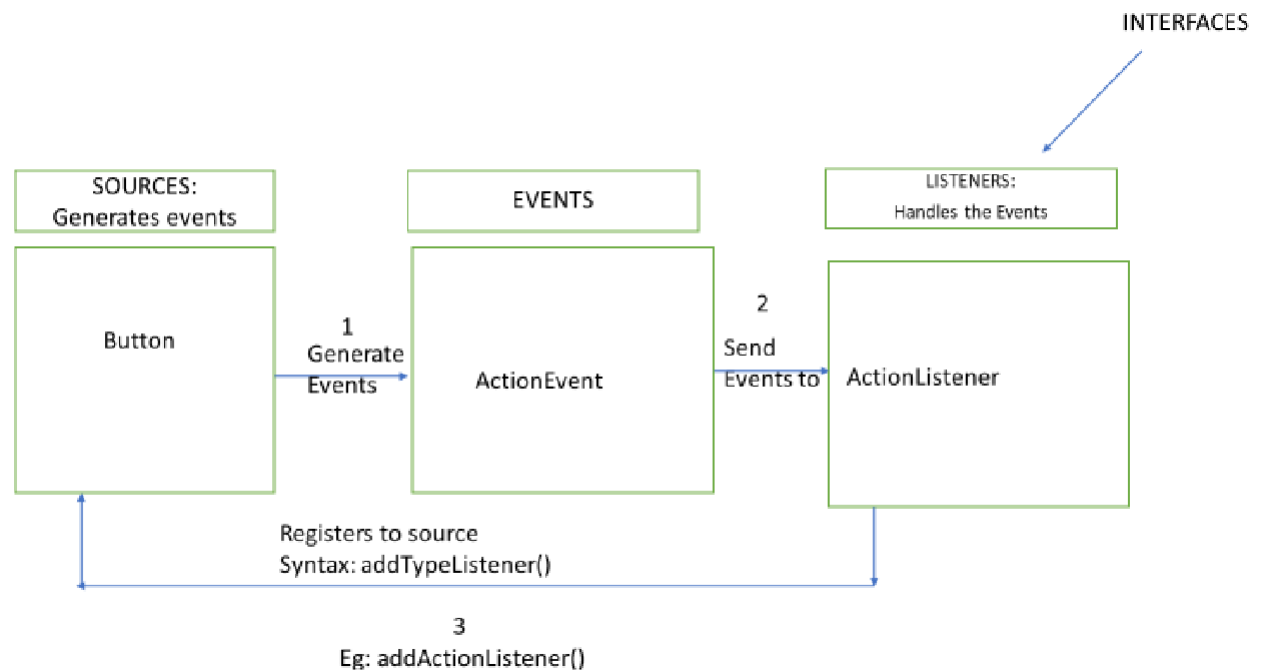
- A source generates an Event and send it to one or more listeners registered with the source.
- Once event is received by the listener, they process the event and then return.
- Events are supported by a number of Java packages, like java.util, java.awt and java.awt.event
- This is an efficient way of handling the event because the event notifications are sent only to those listeners that want to receive them.



Following are the Steps to Handle Events

1. To Declare and instantiate the event sources (or components) as buttons, menus, choices etc.
2. To Implement an interface (listener) for providing the event handler that responds to event source activity.
3. To Register this event handler with event source.
4. To be able to respond to events, a program must:
 - i. Create an event listener object for the type of event.
 - ii. Register the listener object with the GUI component that generates the event (or with a component that contains it).
 - iii. In the picture, the component is the button, contained in a frame. The user event is a click on that button. An Event object is sent to the registered listener. This is done by the Java system, which manages the GUI components. Now it is the responsibility of the listener to do something in response to the event.





18 (b) Write a simple program by extending appropriate class to demonstrate the working of threads in java?

Creating a Thread

- Threads can be created by using two mechanisms:
 - Extending the Thread class
 - Implementing the Runnable Interface

Thread class:

- Thread class provide constructors and methods to create and perform operations on a thread.
- Commonly used **Constructors** of Thread class:
 - Thread ()
 - Thread (String name)
 - Thread (Runnable r)
 - Thread (Runnable r, String name)
- Thread class extends Object class and implements Runnable interface.
- Commonly used **methods** of Thread class:
 - public void run (): is used to perform action for a thread.
 - public void start (): starts the execution of the thread.
 - JVM calls the run () method on the thread

2. By extending the thread class

- Create a new class that extends Thread and create an instance of that class.
- The extending class must override the run() method → entry point for the new thread
- It must also call start() → to begin execution of the new thread.

Steps

1. Use a run() method which is initiated by start() method

```

class A extends Thread
{
    public void run()
  
```

```

    {
        -----
    }
}

```

2. Invoking the run() method

- Creating the object of the corresponding thread class in the main() method class.
Thread_classname objName = new Thread_classname();
objName.start(); // invokes run() method

Java Thread Example by extending Thread class

```

class Multi extends Thread
{
    public void run()
    {
        System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        Multi t1=new Multi();
        t1.start();
    }
}

```

OUTPUT

thread is running...

2. Implementing the Runnable Interface

- **Steps for using a Runnable interface in Java**
 - Create a class that implements the Runnable interface.
 - override the run() method in the Runnable class.
 - need to pass the Runnable object as a parameter to the constructor of the Thread class object while creating it.
 - Then this Thread object is capable of executing our Runnable class.
 - Finally, need to invoke the Thread object's start() method.

Java Thread Example by implementing Runnable interface

```

class Multi3 implements Runnable
{
    public void run ()
    {
        System.out.println("thread is running...");
    }
    public static void main (String args []) {
        Multi3 m1=new Multi3();
        Thread t1 =new Thread(m1);
        t1.start(); }}

```

Output:

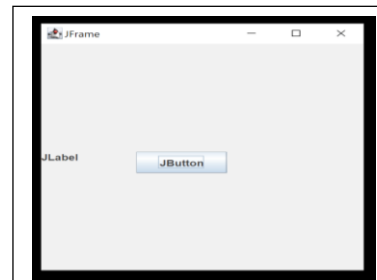
thread is running...

19. (a) Write a Java program to demonstrate the use of JLabel and JButton by adding them to JFrame.

```

import javax.swing.*;
public class JFrameExample {
    public static void main(String s[]) {
        JFrame frame = new JFrame("JFrame");
        JLabel label = new JLabel("JLabel",JLabel.LEFT);
        JButton b=new JButton("JButton");
    }
}

```



```
b.setBounds(100,150,95,30);
frame.add(b);
frame.add(label);
frame.setSize(350, 350);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true); }}
```

19(b) Explain step-by-step procedure of using Java DataBase Connectivity in Java programs.

Java DataBase Connectivity (JDBC)

- JDBC stands for Java Database Connectivity, which is a standard Java **API (Application Programming Interface)** for database-independent connectivity between the Java programming language and a wide range of databases.

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

1. **Register the Driver class**
 2. **Create connection**
 3. **Create statement**
 4. **Execute queries**
 5. **Close connection**
1. **Register the Driver class**
 - The **forName()** method is used to register the driver class.
 2. **Create connection**
 - The **getConnection()** method of DriverManager class is used to establish connection with the database.
 3. **Create statement**
 - The **createStatement()** method of Connection interface is used to create statement.
 - The object of statement is responsible to execute queries with the database.
 4. **Execute queries**
 - The **executeQuery()** method of Statement interface is used to execute queries to the database.
 - This method returns the object of ResultSet that can be used to get all the records of a table.
 5. **Close connection**
 - By closing connection object statement and ResultSet will be closed automatically.
 - The **close ()** method of Connection interface is used to close the connection.

Java Database Connectivity with MySQL

- To connect Java application with the MySQL database, we need to follow 5 following steps.
- In this example we are using MySQL as the database.
- So need to know following information's for the MySQL database:
 1. **Driver class:** The driver class for the MySQL database is com.mysql.jdbc.Driver.
 2. **Connection URL:** The connection URL for the mysql database is jdbc:mysql://localhost:3306/sonoo where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and sonoo is the database name. We may use any database, in such case, we need to replace the sonoo with our database name.
 3. **Username:** The default username for the mysql database is root.
 4. **Password:** It is the password given by the user at the time of installing the mysql database.

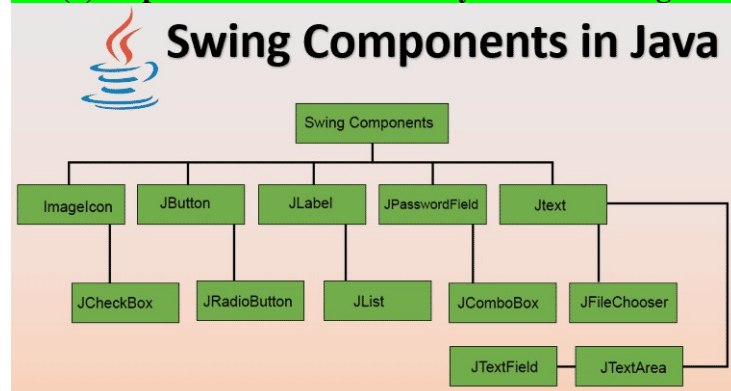
Example to Connect Java Application with MySQL database

- In this example, sonoo is the database name, root is the username and password both.
- First create a table in the mysql database, but before creating table, we need to create database first.
 - create database sonoo;
 - use sonoo;
 - create table emp(id int(10),name varchar(40),age int(3));

```
import java.sql.*;
class MysqlCon
{
    public static void main(String args[])
    {
        try
        {
            //STEP 1: Register the Driver class
            Class.forName("com.mysql.jdbc.Driver");
            //STEP 2: Create connection
            Connection con=DriverManager.getConnection( "jdbc:mysql://localhost:3306/sonoo","root","root");
            //here sonoo is database name, root is username and password
            //STEP 3: Create statement
            Statement stmt=con.createStatement();
            //STEP 4: Execute queries
            ResultSet rs=stmt.executeQuery("select * from emp");
            while(rs.next()) // Extract data from result set
            System.out.println(rs.getString(1)+ " "+rs.getInt(2));
            //STEP 5: Close connection
            con.close();
        } catch(Exception e){ System.out.println(e);}
    }
}
```

emp	
Name	Age
Poornima	29
Sangeeth	32
Pratheesh	30

20. (a) Explain the class hierarchy of Java Swing components?



Swing components are the basic building blocks of an application.

1. JLabel

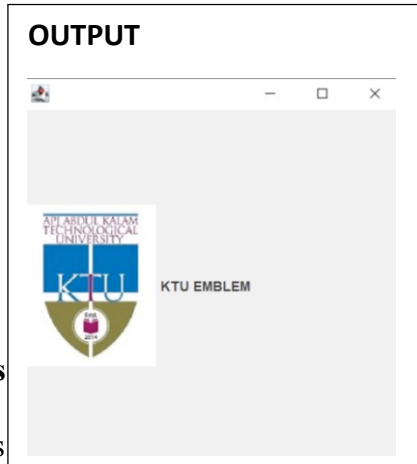
JLabel Constructors used are:

- JLabel(Icon ic)
- JLabel(String str)
- JLabel(String str,Icon ic,int align)
 - Here Icon is abstract class that cannot be instantiated.
 - ImageIcon is a class that extends Icon.
 - So to load images the following statement can be used:
ImageIcon ic=new ImageIcon("filename");
where filename is a string quantity.

JLabel Example

```
import javax.swing.*;
public class SimpleLabel extends JFrame
{
    SimpleLabel()
    {
```

```
        ImageIcon ic=new ImageIcon("C:\\Users\\kiran\\OneDrive\\Pictures\\Desktop\\KTU.jpg");
        JLabel jl=new JLabel("KTU EMBLEM",ic,JLabel.LEFT);
        setSize(350,350);
        setVisible(true);
        add(jl);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String args[])
    {
        new SimpleLabel();    }
}
```



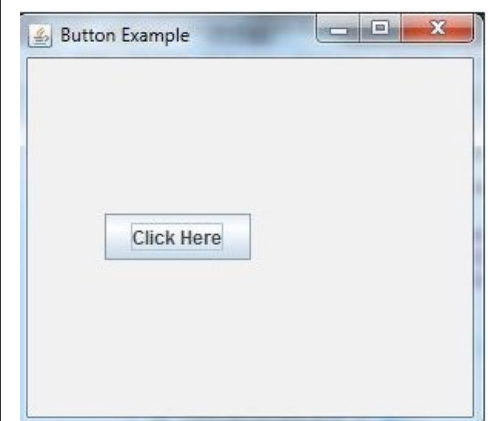
2. JButton

- **Swing JButton**
- **JButton class provides functionality of a button. It is used to create buttons in a Java Swing window.**
 - Some of the methods is setBounds(x,y,w,h)
 - The **setBounds()** method needs four arguments
 - The first two arguments are **x** and **y** coordinates of the **top-left** corner of the component, the **third** argument is the **width** of the component and the **fourth** argument is the **height** of the component.
 - Some of its constructors are shown here:
 - JButton(Icon i)
 - JButton(String s)
 - JButton(String s, Icon i)
 - Here, s and i are the string and icon used for the button

Java JButton Example

```
import javax.swing.*;
public class ButtonExample
{
    public static void main(String[] args)
    {
        JFrame f=new JFrame("Button Example");
        JButton b=new JButton("Click Here");
        b.setBounds(50,100,95,30);
        f.add(b);
    }
}
```

OUTPUT



```

        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } }

```

3. **Java JTextField**

The object of a JTextField class is a text component that allows the editing of a single line text. It inherits JTextComponent class.

Commonly used Constructors:

Constructor	Description
JTextField()	Creates a new TextField
JTextField(String text)	Creates a new TextField initialized with the specified text.
JTextField(String text, int columns)	Creates a new TextField initialized with the specified text and columns.
JTextField(int columns)	Creates a new empty TextField with the specified number of columns.

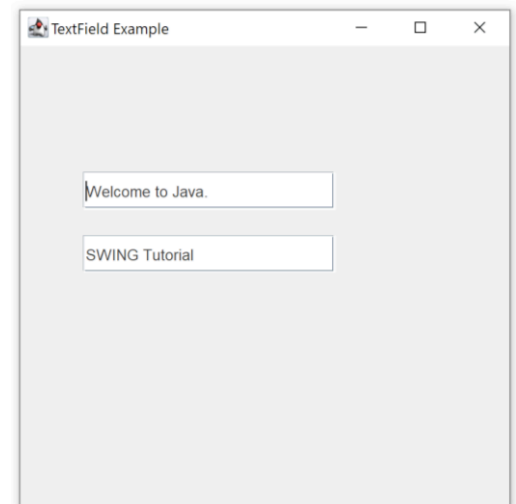
Java JTextField Example

```

import javax.swing.*;
class TextFieldExample
{
public static void main(String args[])
{
    JFrame f= new JFrame("TextField Example");
    JTextField t1,t2;
    t1=new JTextField("Welcome to Java.");
    t1.setBounds(50,100, 200,30);
    t2=new JTextField("SWING Tutorial");
    t2.setBounds(50,150, 200,30);
    f.add(t1);
    f.add(t2);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } }

```

OUTPUT



20(b) Write a Java Program to create a student table and to add student details to it using JDBC.

```

import java.sql.*;
class MysqlCon
{
    public static void main(String args[])
    {
        try

```



```

{
// STEP 1: Register the Driver class
Class.forName("com.mysql.jdbc.Driver");
//STEP 2: Create connection
Connection con=DriverManager.getConnection( "jdbc:mysql://localhost:3306/sonoo","root","
root");
//here sonoo is database name, root is username and password
//STEP 3: Create statement
Statement stmt=con.createStatement();
//STEP 4: Execute queries
Create a table named "student".
stmt.execute("CREATE TABLE student (Roll_no int, Name varchar (31),Age int)");

Inserting data into table named "student".
String sql = "INSERT INTO student +"VALUES (100, 'Zara',18)";
stmt.executeUpdate(sql);
    sql = "INSERT INTO student " + "VALUES (101, 'Mahnaz', 25)";
    stmt.executeUpdate(sql);
    sql = "INSERT INTO student " +"VALUES (102, 'Zaid', 30)";
    stmt.executeUpdate(sql);

//STEP 5: Close connection
con.close();
} catch(Exception e){ System.out.println(e);}
}
}

```

SEMESTER :3

CST 205: OBJECT ORIENTED PROGRAMMING USING JAVA

MODEL QUESTION SET – II

PART A

1. What are the advantages of using UML?

Answer:

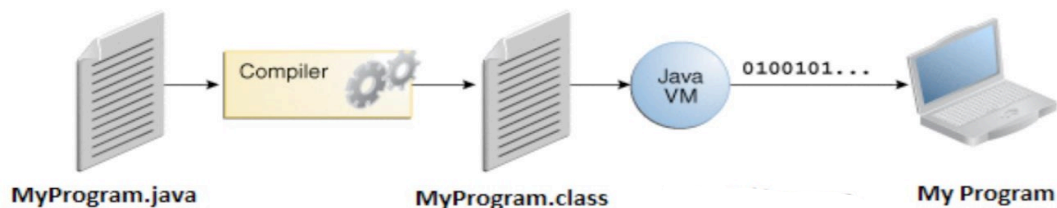
- Provides standard for software development.
- Reducing of costs to develop diagrams of UML using supporting tools.
- Development time is reduced.
- The past faced issues by the developers are no longer exists.
- Has large visual elements to construct and easy to follow.

2. Illustrate the steps involved in JAVA compilation.

Answer:

Compile and Run Java Program

In the Java programming language, all source code is first written in plain text files and save with the **.java** extension. After compilation, **.class** files are generated by **javac** compiler. A **.class** file does not contain code that is native to your processor; it instead contains bytecodes (it is machine language of the Java Virtual Machine1 (JVM)).



The java launcher tool then runs your application with an instance of the Java Virtual Machine (JVM).

3. What is the difference between constructor and method in java?

Answer:

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviour of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

4. What is the purpose of 'this' keyword in Java?

Answer:

- 'this' keyword can be used inside any method to refer to the **current object**.
- **this** is always a reference to the object on which the method was invoked.
- **this** can be used to refer current class instance variable.
- **this** can be used to invoke current class method (implicitly)
- **this()** can be used to invoke current class constructor.
- **this** can be passed as an argument in the method call.
- **this** can be passed as argument in the constructor call.

Java program illustrating the use of this keyword in Java

```
class Area
{
    int length, breadth;
    Area (int l, int b)
    {
        System.out.println("Parameterized Constructor is automatically called");
        this.length = l;
        this.breadth = b;
    }
    void calc()
    {
        int A = length * breadth;
        System.out.println(A);
    }
}
class BoxDemo
{
    public static void main(String args[])
    {
        Area mybox1 = new Area(20,30);
        mybox1.calc(); } }
```

OUTPUT

```
Parameterized Constructor is automatically called
600
```

5. What is the use of interface in Java? Give example?

Answer

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- The interface in Java is *a mechanism to achieve* abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

Usage of Java interface

- It is used to achieve abstraction.
- By interface, can be used to support the functionality of multiple inheritance.

Syntax:

```
interface <interface_name>
{
    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

Java Interface Example

In this example, the Printable interface has only one method, and its implementation is provided in the A6 class.

interface printable

```
{  
    void print();  
}
```

class A6 implements printable

```
{  
    public void print()  
    {  
        System.out.println("Hello");  
    }  
}
```

Class Demo

```
{  
    public static void main(String args[])  
    {  
        A6 obj = new A6();  
        obj.print();    } }  
}
```

Output:
Hello

6. What are packages? Write the steps involved in creating a package in java?

Answer

- A java package is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- **Advantage of Java Package**
 - a. Java package is used to categorize the classes and interfaces so that they can be easily maintained.
 - b. Java package provides access protection.
 - c. Java package removes naming collision.
 - d. Reusability of code

Steps involved in creating a package

1. Declare the package at the beginning of the file using 'package' keyword
 - a. Syntax : package package_name;
2. Define a class inside the package and declare it as a public class.
3. Create a subdirectory under the directory where the source files are stored.
4. Store the classname.java file in the subdirectory.
5. Compile the file. It creates a.class file and is stored in the subdirectory.

7. Write any three constructors of String class.

Answer:

STRING CONSTRUCTORS

- The string class supports several types of constructors in Java APIs. The most commonly used constructors of String class are as follows:
1. **String ()**: To create an empty String, we will call a default constructor.
 2. **String (String str)**: It will create a string object in the heap area and stores the given value in it.

3. **String (char chars [])**: It will create a string object and stores the array of characters in it.
-

8. **What are the advantages and disadvantages of multithreading in Java?**

Answer:

Advantages of Multithreading:

1. Enhanced performance by decreased development time.
2. Simplified and streamlined program coding.
3. Improvised GUI responsiveness
4. Simultaneous and parallelized occurrence of tasks
5. Better use of cache storage by utilization of resources
6. Decreased cost of maintenance.
7. Better use of CPU resource

Disadvantages of Multithreading:

1. Complex debugging and testing processes
 2. Overhead switching of context
 3. Increased potential for deadlock occurrence
 4. Increased difficulty level in writing a program
 5. Unpredictable results
-

9. **What are the advantages of using Swing API?**

Answer:

- Swing provides both additional components and added functionality to AWT-replacement components
 - Swing components can change their appearance based on the current "look and feel" library that's being used. You can use the same look and feel as the platform you're on, or use a different look and feel
 - Swing components follow the Model-View-Controller paradigm (MVC), and thus can provide a much more flexible UI.
 - Swing provides "extras" for components, such as:
 - Icons on many components
 - Decorative borders for components
 - Tooltips for components
 - Swing components are lightweight (less resource intensive than AWT)
 - Swing provides built-in double buffering
 - Swing provides paint debugging support for when you build your own components
-

10. **Explain JDBC Architecture?**

Answer:

- JDBC Architecture consists of two layers

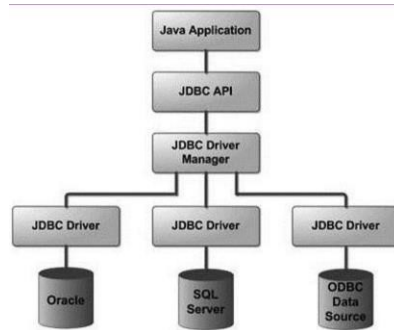
1. **JDBC API:**

- This provides the application-to-JDBC Manager connection.
- The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.

2. **JDBC Driver API:**

- This supports the JDBC Manager-to-Driver Connection.
- The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.
- The JDBC driver manager ensures that the correct driver is used to access each data source.

- Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application



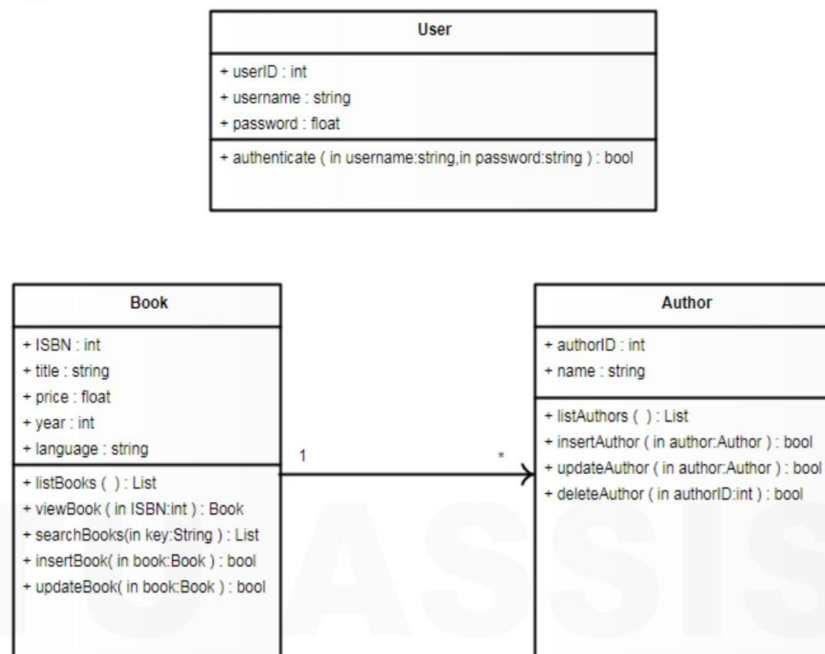
PART B

11. Represent the following entities using UML class diagram.

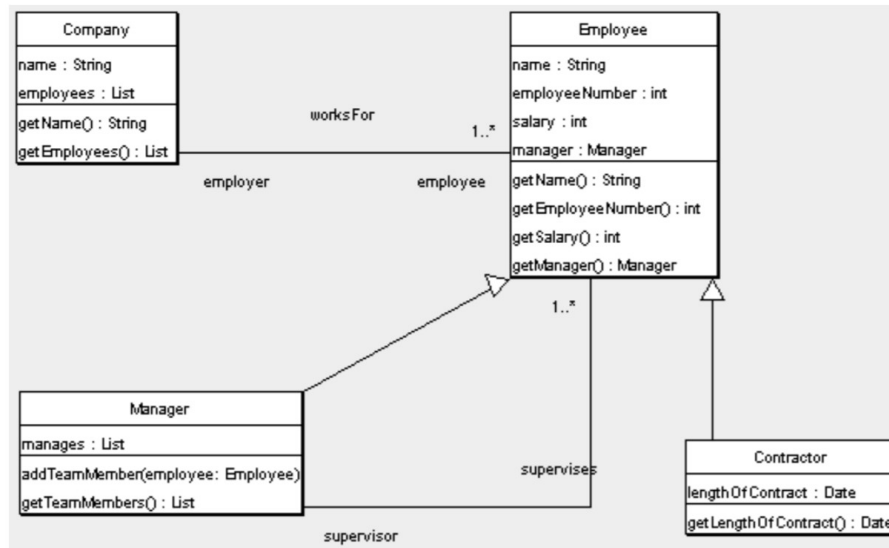
- a. Book b. Employee c. Vehicle

Answer:

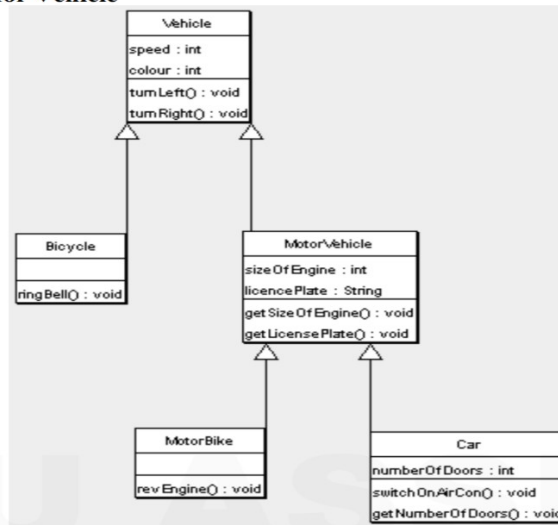
(i) Class Diagram for Book



(ii) Class Diagram for Employee

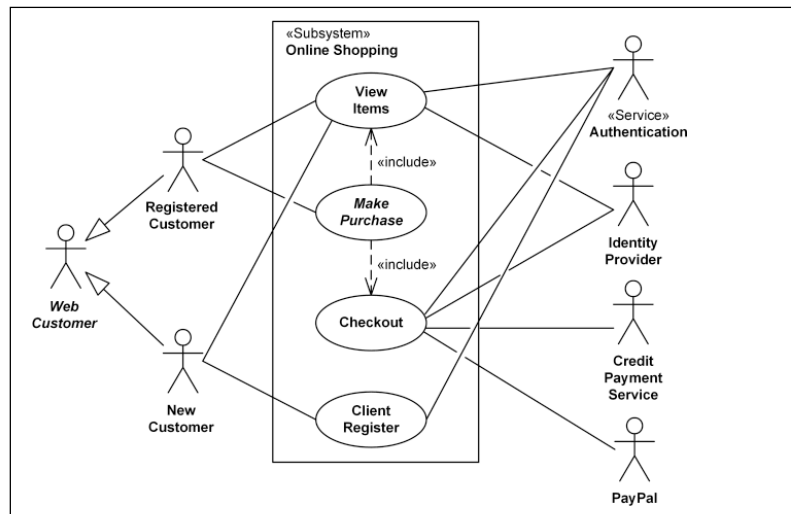


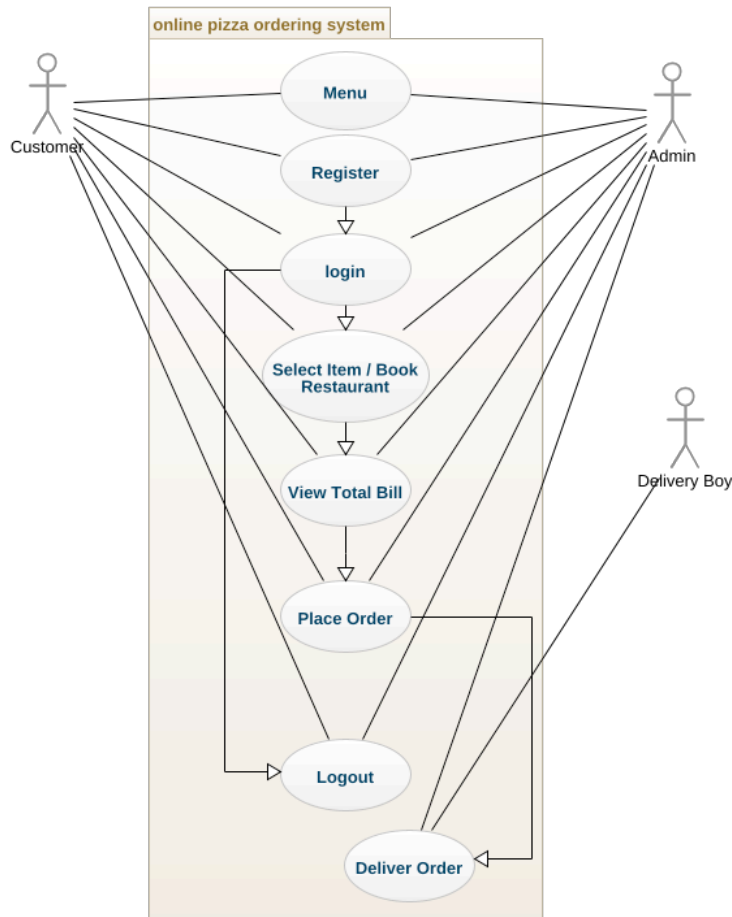
(iii) Class Diagram for Vehicle



12. Construct Use-case diagram for
- An Online Shopping Application.
 - Online Pizza ordering system

Answer:





13. Describe the types of constructor in Java, with suitable examples?

Answer:

- A constructor **helps to initialize an object** (give values) immediately upon creation.
- Constructor is a special method inside the class.
- Constructor has the same name as the class in which it resides.

Two types of constructors

1. Default constructor – has no arguments
2. Parameterized constructor –has arguments(parameters)

1. **Default constructor:** The constructors having no arguments or parameters.

E.g.

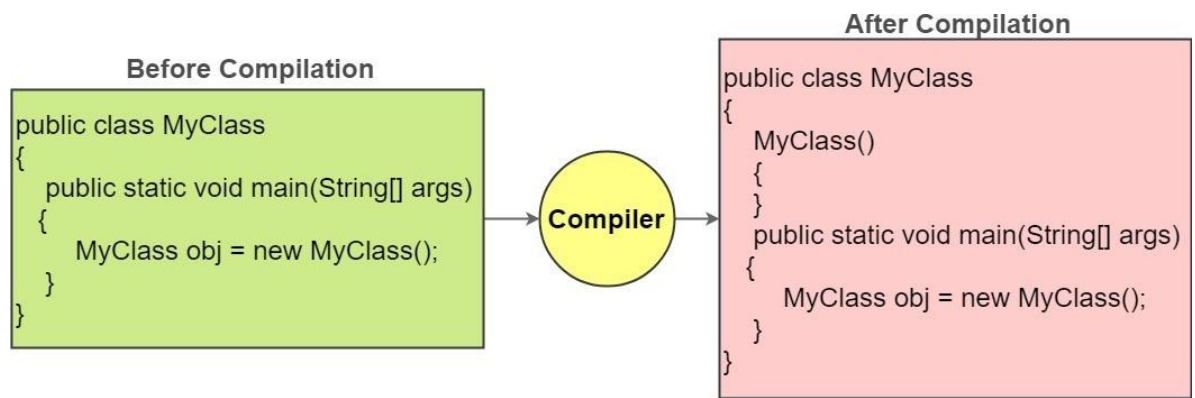
class A

```

{
    A ();
    {
        //statements
    }
}

```

Default Constructor of class A



- When we do not explicitly define a constructor for a class, then **Java creates a default constructor for the class.**

Java program illustrating the use of Default constructor in Java

```

class Box
{
    Box ()
    {
        System.out.println("Constructor is calling");
    }
}
class BoxDemo
{
    {
        public static void main (String args [])
        {
            Box mybox1 = new Box ();
            Box mybox2 = new Box ();
        }
    }
}

```

OUTPUT

```

Constructor is calling
Constructor is calling

```

Write a program to find the area of a rectangle using constructor.

```

class Area
{
    int length,breadth;
    Area()
    {
        System.out.println("Constructor is automatically called");
        length = 20;
        breadth = 3;
    }
    void calc()
    {
        int A = length * breadth;
        System.out.println(A);
    }
}
class BoxDemo
{
    {
        public static void main(String args[])
        {

```

OUTPUT

```

Constructor is automatically called
60

```

```

        Area mybox1 = new Area();
        mybox1.calc();
    }
}

```

2. **Parameterized Constructors**

- Constructors with arguments are called parameterized constructors.

SYNTAX

```

class A
{
    A (arg1, arg2, ..., argn)
    {
        // constructor body
    }
}

```

Write a program to find the area of a rectangle using Parameterized constructor.

PROGRAM

```

class Area
{
    int length, breadth;
    Area (int l, int b)
    {
        System.out.println("Parameterized Constructor is automatically called");
        length = l;
        breadth = b;
    }
    void calc ()
    {
        int A = length * breadth;
        System.out.println(A);
    }
}
class BoxDemo
{
    public static void main (String args [])
    {
        Area mybox1 = new Area (20,30);
        mybox1.calc();
    }
}

```

OUTPUT

Parameterized Constructor is automatically called
600

14. **Discuss the different access specifiers used in java.**

Answer:

ACCESS CONTROL KEYWORDS OR ACCESS MODIFIERS IN JAVA

- How a member can be accessed is determined by the access modifier attached to its declaration.
- Java's access modifiers are public, private, and protected.
- The four access levels are –

- Visible to the package, the **default**. No modifiers are needed.
- Visible to the class only (**private**).
- Visible to the world (**public**).
- Visible to the package and all subclasses (**protected**).
- When no access modifier is used, then by default the member of a class is public within its own package but cannot be accessed outside of its package.

	PRIVATE	DEFAULT	PROTECTED	PUBLIC
1 Same class	Yes	Yes	Yes	Yes
2 Same package Subclass	No	Yes	Yes	Yes
3 Same package Non-subclass	No	Yes	Yes	Yes
4 Different package Subclass	No	No	Yes	Yes
5 Different package Non-subclass	No	No	No	Yes

15. What is an exception? What are the categories of exceptions? Also discuss the advantages of exception handling in Java.

Answer:

What is Exception?

- Exception is an abnormal condition.
- In Java, an exception is an event that disrupts the normal flow of the program.
- It is an object which is thrown at runtime.

What is Exception Handling?

- The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that normal flow of the application can be maintained.
- Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.
- The core advantage of exception handling is to maintain the normal flow of the application.
- An exception normally disrupts the normal flow of the application that is why we use exception handling.

Types of Java Exceptions

- There are mainly two types of exceptions:
 - checked and
 - unchecked.

- Here, an error is considered as the unchecked exception.
- According to Oracle, there are three types of exceptions:

1. **Checked Exception**
2. **Unchecked Exception**
3. **Error**

1. **Checked Exception**

- The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions.
- e.g. IOException, SQLException etc.
- Checked exceptions are **checked at compile-time**.

2. **Unchecked Exception**

- The classes which inherit RuntimeException are known as unchecked exceptions
- e.g. ArithmeticException, NullPointerException,
- Unchecked exceptions are not checked at compile-time, but they are **checked at runtime**.

3. **Error**

- Error is irrecoverable
- e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

There are 5 keywords which are used in handling exceptions in Java.

Keyword	Description
Try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
Catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
Finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
Throw	The "throw" keyword is used to throw an exception.
Throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

16. **Discuss the concept of Interfaces and the types of interfaces in Java with example.**

Answer:

INTERFACE IN JAVA

- Interface is a pure abstract class.

- An abstract class is one in which there is a declaration but no definition for a member function.
- A **pure Abstract class** has only abstract member functions and no data or member functions.
- They are syntactically like classes.
- They can't create instance (object) of an interface.
- Methods of interface are declared without any body.
 - **Methods** in the interface will be **public**.
 - **Data members** are **public, static** and **final**.

Syntax:

```
interface interface_name
{
    variable declaration;
    method declaration;    // abstract methods having declarations only
}
```

Variable declaration:

```
static final type variable_name = value;
```

- type : datatype can be int, float.....

Eg: static final int A = 10;

Method Declaration :

```
return_type method_name(parameter list);
```

- Return_type : data type can be int, float.....
- Have only declaration and no body

Eg: int sum(int a , int b);

Example:

```
interface Area
{
    final static float pi =3.14;
    float compute (float x, float y);
}
```

Why use Java interface?

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

Types Of Interface

1. Interface can also be extended (Extending Interfaces)

Syntax:

```
interface child_class extends parent_class
{
    body of child_class
}
```

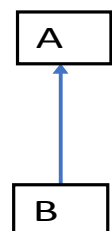
2. Implementing Interfaces

Syntax:

```
class class_name implements interface_name
{
    body of class_name
}
```

Example:

```
interface B extends A
{
    body of B
}
```



```
}
```

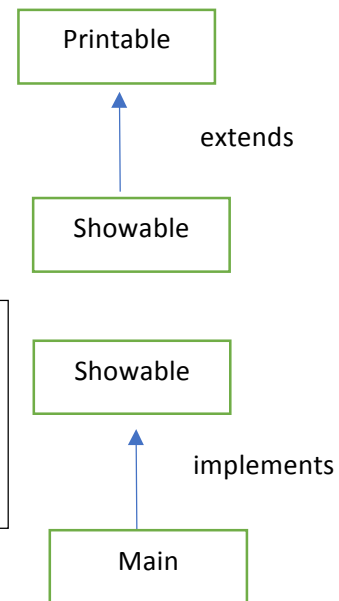
PROGRAM

A class implements an interface, but one interface extends another interface.

```
interface Printable
{
    void print();
}
interface Showable extends Printable
{
    void show();
}
class Main implements Showable
{
    public void print()
    {
        System.out.println("Hello");
    }
    public void show()
    {
        System.out.println("Welcome");
    }
    public static void main(String args[])
    {
        Main obj = new Main();
        obj.print();
        obj.show();
    }
}
```

OUTPUT

```
Hello
Welcome
```

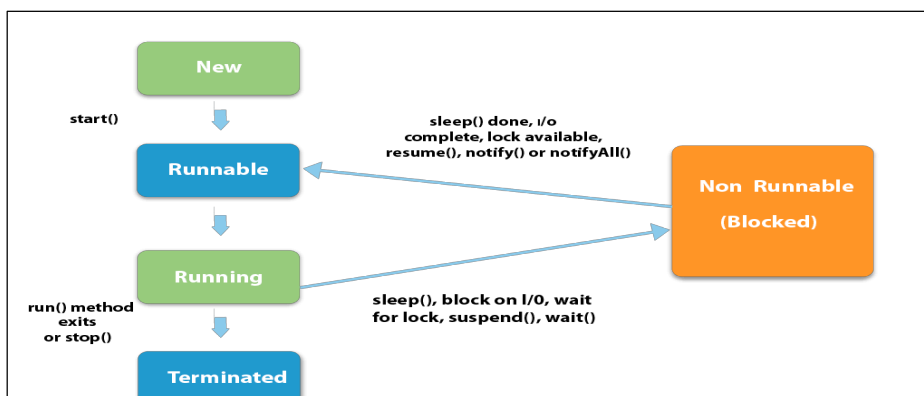


17. Discuss the thread model / life cycle with suitable figure?

Answer:

Life cycle of a Thread (Thread States)/Java Thread Model

- A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies.
- A thread can be in one of the five states.
- The life cycle of the thread in java is controlled by JVM.
- The java thread states are as follows:
 1. New
 2. Runnable
 3. Running
 4. Non-Runnable (Blocked)
 5. Terminated



- 1) New: The thread is in new state if you create an instance of Thread class but before the invocation of start () method.
- 2) Runnable: The thread is in runnable state after invocation of start () method, but the thread scheduler has not selected it to be the running thread.
- 3) Running: The thread is in running state if the thread scheduler has selected it.
- 4) Non-Runnable (Blocked): This is the state when the thread is still alive but is currently not eligible to run.
- 5) Terminated: A thread is in terminated or dead state when its run() method exits.

18. Write a java program to create two threads, one for writing odd numbers and another for writing even numbers up to 100 in two different files.

ANSWER:

```
class Runnable1 implements Runnable
{
    public void run()
    {
        for (int i=0;i<100;i++)
        {
            if (i%2 == 1)
                System.out.println("ODD Numbers are:" +i );
        }
    }
}

class Runnable2 implements Runnable
{
    public void run()
    {
        for (int i=0;i<100;i++)
        {
            if (i%2 == 0)
                System.out.println("EVEN Numbers are:" +i );
        }
    }
}

public class Mythread
{
    public static void main(String[] args)
    {
        Runnable r = new Runnable1();
        Thread t = new Thread(r);
        t.start();
        Runnable r2 = new Runnable2();
```

```
Thread t2 = new Thread(r2);
t2.start();    } }
```

19. What are layout managers? Explain any one layout manager with an example?

Answer:

- Layout refers to the arrangement of components within the container.
- The task of laying out the controls is done automatically by the Layout Manager.
 - Automatically positions all the components within the container.
- Properties like size, shape, and arrangement varies from one layout manager to the other.
- LayoutManager is an interface which implements the classes of the layout manager.
- There are following classes that represents the layout managers:
 1. java.awt.GridLayout
 2. java.awt.BorderLayout
 3. java.awt.FlowLayout
 4. javax.swing.BoxLayout
 5. javax.swing.GroupLayout
 6. javax.swing.ScrollPaneLayout
 7. javax.swing.SpringLayout

2. Java GridLayout

- The GridLayout is used to arrange the components in rectangular grid. One component is displayed in each rectangle.
- Constructors of GridLayout class
 1. **GridLayout():** creates a grid layout with one column per component in a row.
 2. **GridLayout(int rows, int columns):** creates a grid layout with the given rows and columns but no gaps between the components.
 3. **GridLayout(int rows, int columns, int hgap, int vgap):** creates a grid layout with the given rows and columns alongwith given horizontal and vertical gaps.

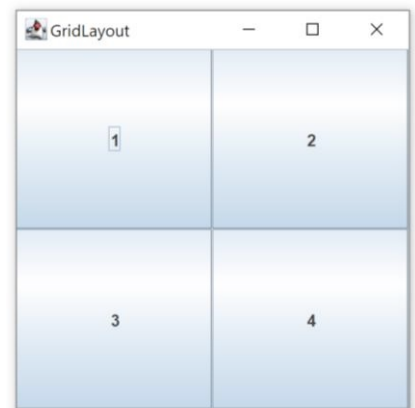
Java program illustrating the GridLayout

```
import java.awt.*;
import javax.swing.*;
public class MyGridLayout{
MyGridLayout()
{
    JFrame f=new JFrame();
    JButton b1=new JButton("1");
    JButton b2=new JButton("2");
    JButton b3=new JButton("3");
    JButton b4=new JButton("4");

    f.add(b1);
    f.add(b2);
    f.add(b3);
    f.add(b4);

    f.setLayout(new GridLayout(2,2));
    //setting grid layout of 2 rows and 2 columns
```

OUTPUT




```

        f.setSize(300,300);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        new MyGridLayout();
    }
}

```

20. Discuss the different steps involved in establishing a JDBC connectivity and query processing with a suitable example.

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

1. **Register the Driver class**
 2. **Create connection.**
 3. **Create statement.**
 4. **Execute queries.**
 5. **Close connection**
1. **Register the Driver class**
 - The **forName()** method is used to register the driver class.
 2. **Create connection**
 - The **getConnection()** method of DriverManager class is used to establish connection with the database.
 3. **Create statement**
 - The **createStatement()** method of Connection interface is used to create statement.
 - The object of statement is responsible to execute queries with the database.
 4. **Execute queries**
 - The **executeQuery()** method of Statement interface is used to execute queries to the database.
 - This method returns the object of ResultSet that can be used to get all the records of a table.
 5. **Close connection**
 - By closing connection object statement and ResultSet will be closed automatically.
 - The **close ()** method of Connection interface is used to close the connection.

Java Database Connectivity with MySQL

- To connect Java application with the MySQL database, we need to follow 5 following steps.
- In this example we are using MySQL as the database.
- So need to know following information's for the MySQL database:
 1. **Driver class:** The driver class for the MySQL database is com.mysql.jdbc.Driver.
 2. **Connection URL:** The connection URL for the mysql database is jdbc:mysql://localhost:3306/sonoo where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and sonoo is the database name. We may use any database, in such case, we need to replace the sonoo with our database name.
 3. **Username:** The default username for the mysql database is root.

4. **Password:** It is the password given by the user at the time of installing the mysql database.

Example to Connect Java Application with MySQL database

- In this example, sonoo is the database name, root is the username and password both.
- First create a table in the mysql database, but before creating table, we need to create database first.
 - create database sonoo;
 - use sonoo;
 - create table emp(id int(10),name varchar(40),age int(3));

```
import java.sql.*;
class MysqlCon
{
    public static void main(String args[])
    {
        try
        {
            //STEP 1: Register the Driver class
            Class.forName("com.mysql.jdbc.Driver");
            //STEP 2: Create connection
            Connection con=DriverManager.getConnection( "jdbc:mysql://localhost:3306/sonoo","root",
            "root");
            //here sonoo is database name, root is username and password
            //STEP 3: Create statement
            Statement stmt=con.createStatement();
            //STEP 4: Execute queries
            ResultSet rs=stmt.executeQuery("select * from emp");
            while(rs.next()) // Extract data from result set
            System.out.println(rs.getString(1)+ " "+rs.getInt(2));
            //STEP 5: Close connection
            con.close();
        } catch(Exception e){ System.out.println(e);}
    }
}
```

```
C:\Users\Sangeeth\Desktop\Java>javac Test.java
C:\Users\Sangeeth\Desktop\Java>java Test
Poornima      29
Sangeeth      32
Pratheesh     30
```

emp	
Name	Age
Poornima	29
Sangeeth	32
Pratheesh	30

SEMESTER :3
CST 205: OBJECT ORIENTED PROGRAMMING USING JAVA

MODEL QUESTION SET - III

PART A

1. Write a java program to check whether a given number is prime or not.

ANSWER:

```
public class Prime
{
    public static void main(String[] args)
    {
        int num = 29;
        boolean flag = false;
        for(int i = 2; i <= num/2; ++i)
        {
            if(num % i == 0)
            {
                flag = true;
                break;
            }
        }

        if (!flag)
            System.out.println(num + " is a prime number.");
        else
            System.out.println(num + " is not a prime number.");
    }
}
```

OUTPUT

29 is a prime number.

2. What are bytecodes and JVM? Explain how Java implements Machine Independent Programming?

ANSWER:

- Bytecode is program code that has been compiled from source code into low-level code designed for a software interpreter. ...
- A popular example is Java bytecode, which is compiled from Java source code and can be run on a Java Virtual Machine (JVM).

What is JVM? Java Virtual Machine (JVM) is a engine that provides runtime environment to drive the Java Code or applications. It converts Java bytecode into machines language. JVM is a part of Java Run Environment (JRE). In other programming languages, the compiler produces machine code for a particular system. However, Java compiler produces code for a Virtual Machine known as Java Virtual Machine.

- Platform independent language means once compiled, can execute the program on any platform (OS).
- Java is platform independent. Because the Java compiler converts the source code to bytecode, which is Intermediate Language. Bytecode can be executed on any platform (OS) using JVM(Java Virtual Machine).
- Download JVM's (comes along with JDK or JRE) suitable to any operating system and, write a Java program, can run it on any system using JVM.

3. What is meant by polymorphism? Briefly explain the types of polymorphism.

ANSWER:

- The word polymorphism means having many forms.
- Polymorphism is defined as the ability of a message to be displayed in more than one form.
- Polymorphism is considered as one of the important features of Object Oriented Programming. Polymorphism allows us to perform a single action in different ways. In other words, polymorphism allows you to define one interface and have multiple implementations. The word “poly” means many and “morphs” means forms, So it means many forms.
- In Java polymorphism is mainly divided into two types:
 - Compile time Polymorphism**
 - It is also known as static polymorphism. This type of polymorphism is achieved by **function overloading or operator overloading**.
 - **Method Overloading:** When there are multiple functions with same name but different parameters then these functions are said to be **overloaded**. Functions can be overloaded by **change in number of arguments** or/and **change in type of arguments**.
 - Runtime Polymorphism**
 - It is also known as **Dynamic Method Dispatch**. It is a process in which a function call to the overridden method is resolved at Runtime. This type of polymorphism is achieved by **Method Overriding**.
 - **Method overriding**, on the other hand, occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be **overridden**.

4. Differentiate between Method Overloading and Overriding.

ANSWER:

No.	Method Overloading	Method Overriding
1)	Method overloading is used to <i>increase the readability</i> of the program.	Method overriding is used to <i>provide the specific implementation</i> of the method that is already provided by its super class.
2)	Method overloading is performed <i>within class</i> .	Method overriding occurs <i>in two classes</i> that have IS-A (inheritance) relationship.
3)	In case of method overloading, <i>parameter must be different</i> .	In case of method overriding, <i>parameter must be same</i> .
4)	Method overloading is the example of <i>compile time polymorphism</i> .	Method overriding is the example of <i>run time polymorphism</i> .
5)	In java, method overloading can't be performed by changing return type of the method only. <i>Return type can be same or different</i> in method overloading. But you must have to change the parameter	<i>Return type must be same or covariant</i> in method overriding.

5. What are the different exception keywords in JAVA?

ANSWER:

Keyword	Description
Try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
Catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
Finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
Throw	The "throw" keyword is used to throw an exception.
Throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

6. Write a Java program that read from a file and write to file by handling all file related exceptions using FileReader and FileWriter classes.

ANSWER

```
import java.io.*;
class Main
{
    public static void main(String a[]) throws IOException
    {

        try
        {
            FileReader f1= new FileReader("test.txt");
            FileWriter f2= new FileWriter("cp.txt");
            int c;
```

```

while((c=f1.read())!=-1)
{
    f2.write((char)c);
    System.out.print((char)c);
}
f1.close();
f2.close();
}
catch(FileNotFoundException e)
{
    System.out.println("File not found");
}}

```

OUTPUT

India is my country.

test.txt

India is my country.

cp.txt

India is my country.

7. Differentiate between String and StringBuffer classes.

ANSWER

No.	String	StringBuffer
1)	String class is immutable.	StringBuffer class is mutable.
2)	String is slow and consumes more memory when you concat too many strings because every time it creates new instance.	StringBuffer is fast and consumes less memory when you concat strings.
3)	String class overrides the equals() method of Object class. So it is possible to compare the contents of two strings by equals() method.	StringBuffer class doesn't override the equals() method of Object class.

Using the String Class	Using the StringBuffer Class
<pre> class Main { public static void main(String args[]) { String s1=new String("Sachin"); s1.concat(" Tendulkar"); System.out.println(s1); } } </pre>	<pre> class Main { public static void main(String args[]) { StringBuffer s1=new StringBuffer("Sachin"); s1.append(" Tendulkar"); System.out.println(s1); } } </pre>
<u>OUTPUT</u>	<u>OUTPUT</u>
Sachin	Sachin Tendulkar

CLASS	COMPONENT	INTERFACES
EVENTS	SOURCE	LISTENERS
Action Event	Button, List, MenuItem, Text field	ActionListener
Component Event	Component	Component Listener
Focus Event	Component	FocusListener
Item Event	Checkbox, CheckboxMenuItem, Choice, List	ItemListener
Key Event	when input is received from keyboard	KeyListener
Text Event	Text Component	TextListener
Window Event	Window	WindowListener
Mouse Event	Mouse related event	MouseListener

9. **Differentiate between component and container? List any three Containers and Components available in Swing API.**

ANSWER:

- A component is an independent visual control, such as a push button or slider.
- A container holds a group of components. Thus, a container is a special type of component that is designed to hold other components.
- Swing components inherit from the javax.Swing.JComponent class, which is the root of the Swing component hierarchy
- Swing components are:
 1. JButton
 2. JLabel
 3. JTextField
- Swing Containers are:
 1. JPanel
 2. JFrame
 3. JWindow

10. **Explain MVC architecture?**

ANSWER:

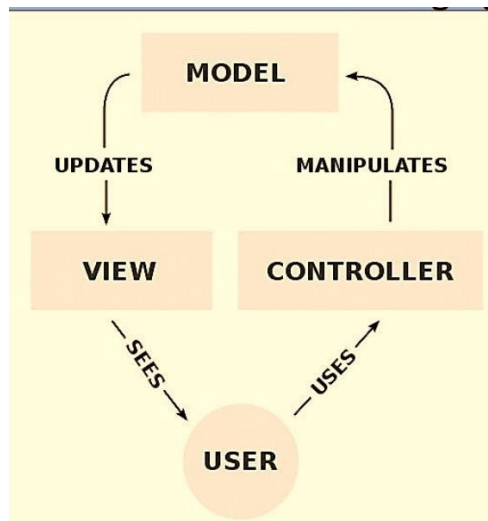
Model-View-Controller Architecture

- Swing uses the model-view-controller architecture (MVC) as the fundamental design behind each of its components
- Essentially, MVC breaks GUI components into three elements.
- Each of these elements plays a crucial role in how the component behaves.
- The Model-View-Controller is a well-known software architectural pattern ideal to implement user interfaces on computers by dividing an application into three interconnected parts

Main goal

- is to separate internal representations of an application from the way's information are presented to the user.
- Initially, MVC was designed for desktop GUI applications but it's quickly become an extremely popular pattern for designing web applications too.
- MVC pattern has the three components:
 - **Model** that manages data, logic and rules of the application
 - **View** that is used to present data to user
 - **Controller** that accepts input from the user and converts it to commands for the Model or View.

MVC pattern defines the interactions between these three components like you can see in the following figure :



- The Model receives commands and data from the Controller.
- It stores these data and updates the View.
- The View lets to present data provided by the Model to the user.
- The Controller accepts inputs from the user and converts it to commands for the Model or the View.

PART B

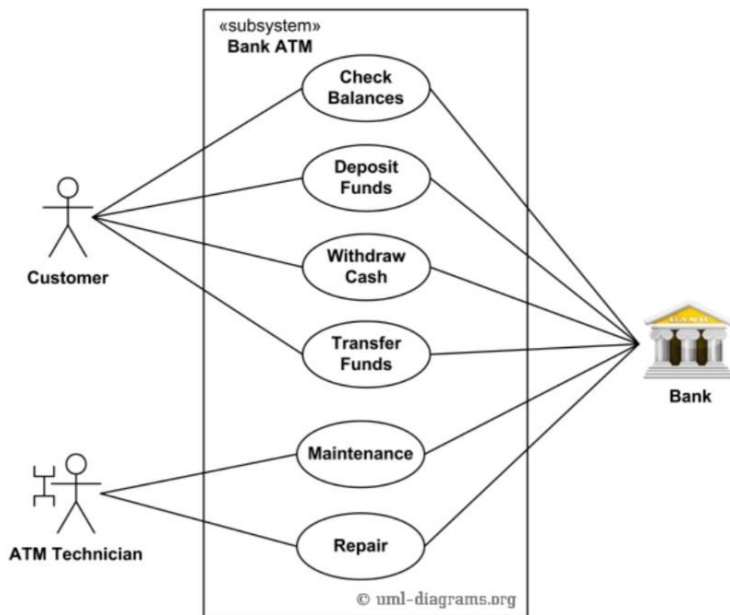
11. Construct Use Case diagrams for the following:

a. ATM b. Library c. Railway reservation

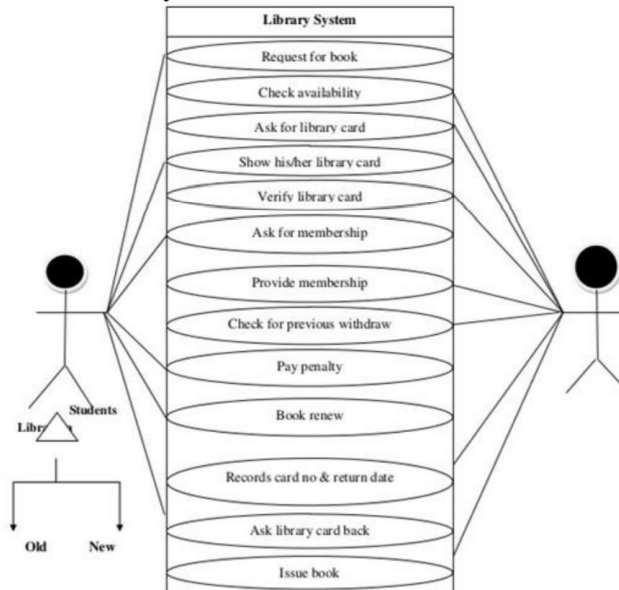
ANSWER:

An automated teller machine (ATM) or the automatic banking machine (ABM) is a banking subsystem (**subject**) that provides bank customers with access to financial transactions in a public space without the need for a cashier, clerk, or bank teller.

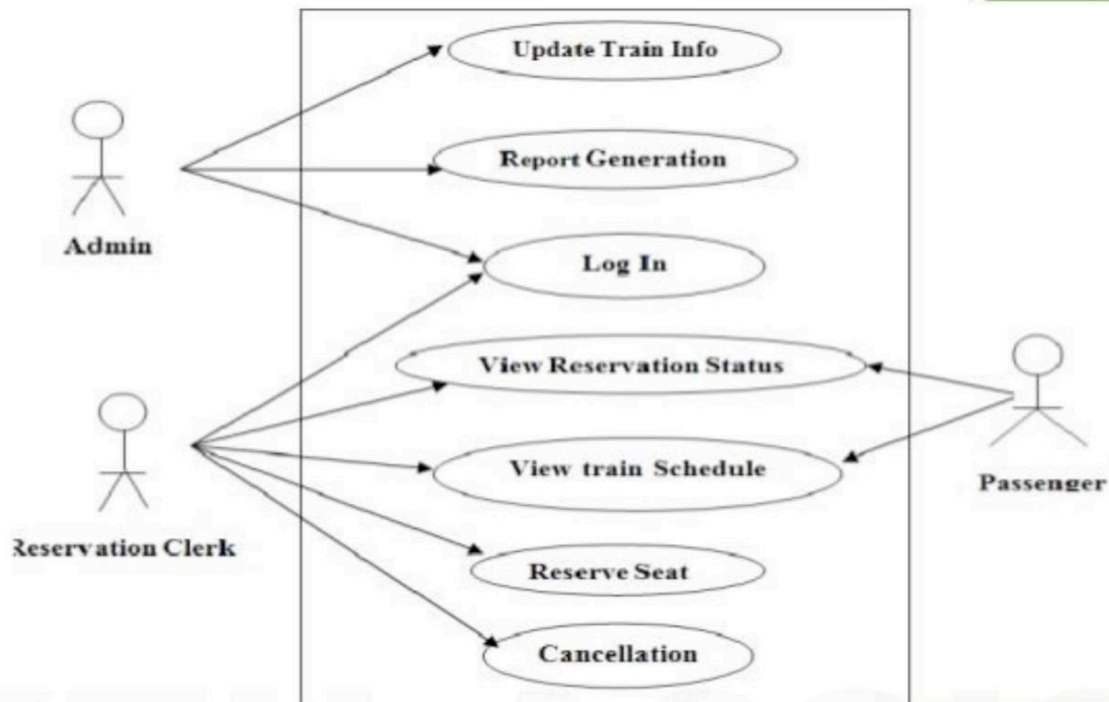
Customer (**actor**) uses bank ATM to *Check Balances* of his/her bank accounts, *Deposit Funds*, *Withdraw Cash* and/or *Transfer Funds* (**use cases**). *ATM Technician* provides *Maintenance* and *Repairs*. All these use cases also involve *Bank* actor whether it is related to customer transactions or to the ATM servicing.



ii) Use Case Diagram for Library



iii) Use Case Diagram for Railway reservation



12. Describe the features of java language? OR Explain Java Buzzwords?

ANSWER:

1. Object Oriented:

- In Java, everything is an Object. Java can be easily extended since it is based on the Object model.

2. Platform Independent:

- Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform-independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.

3. Simple:

- Java is designed to be easy to learn. By understanding the basic concept of OOP Java, it would be easy to master.

4. Secure:

- With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption.

5. Architecture-neutral

- Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.

6. Portable:

- Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. The compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.

7. Robust:

- Java makes an effort to eliminate error-prone situations by emphasizing mainly on compile time error checking and runtime checking.

8. **Multithreaded:**

- With Java's multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.

9. **Interpreted:**

- Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.

10. **High Performance:**

- With the use of Just-In-Time compilers, Java enables high performance.

11. **Distributed:**

- Java is designed for the distributed environment of the internet.

12. **Dynamic:**

- Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry an extensive amount of run-time information that can be used to verify and resolve accesses to objects at run-time.

13. **Explain the Java Program structure with example?**

ANSWER:

Java program structure

Documentation Section
Package Statement
Import Statement
Interface Statement
Class Definition
Main Method Class { //Main method definition }

- **DOCUMENTATION SECTION:** It is a set of comment lines giving the name of the program, the author and the other details, which the programmer would like to refer.
- **PACKAGE STATEMENT:** The first statement allowed in Java files is a package statement. This statement declares a package name and informs the compiler that the classes defined here belong to this package.
 - Ex:package student;
 - It is optional because no need & our class to be part of packages.
- **IMPORT STATEMENT:** Similar to #include statement in C.
 - Ex:import student.test;
 - This statement instructs the interpreter to load the test class contained in package student.

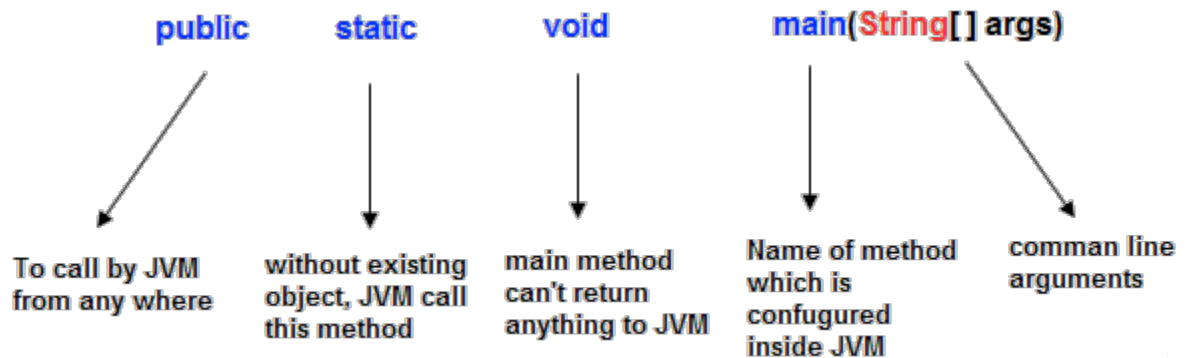
- Using import statement , can access to classes that part of other named packages.
- **INTERFACE STATEMENT:** Interface like class but includes group of methods declaration.
 - Used when we want to implement multiple inheritance feature.
- **CLASS DEFINITION:**
 - Java program may contain multiple class definition.
 - Classes are primary feature of Java program.
 - The classes are used to map real world problems.
- **MAIN METHOD CLASS:**
 - Java stand alone program requires main method as starting point.
 - This is essential part of program
 - Main method creates object of various classes.
 - On reaching end of the main the program terminate and the control passes back to then operating system.
- Note:
 - Java Compiler will compile class that do not contain main() method.
 - If main() is not declared it give runtime error,because at compile time Compiler is not using main() but interpreter use at runtime.

package details	→	import java.io.*
class className	→	class Sum
{		
Data members;	→	int a, b, c;
user_defined method;	→	void display();
public static void main(String args[])		
{		
Block of Statements;	→	System.out.println("Hello Java !");
}		
}		

Tutorial4us.com

- A **package** is a collection of classes, interfaces and sub-packages. A sub package contains collection of classes, interfaces and sub-sub packages etc. java.lang.*; package is imported by default and this package is known as default package.
- **Class** is keyword used for developing user defined data type and every java program must start with a concept of class. "ClassName" represent a java valid variable name treated as a name of the class each and every class name in java is treated as user-defined data type.
- **Data member** represents either instance or static they will be selected based on the name of the class.
- **User-defined methods** represents either instance or static they are meant for performing the operations either once or each and every time.
- Each and every java program starts execution from the **main() method**. And hence main() method is known as program driver.
 - Since main() method must be accessed by every java programmer and hence whose access specifier must be **public**.
 - Since main() method of java executes only once throughout the java program execution and hence its nature must be **static**.
 - Since main() method of java is not returning any value and hence its return type must be **void**.

- Each and every main() method of java must take array of objects of String.
 - String args[] is a String array used to hold command line arguments in the form of String values.
- Block of statements** represents set of executable statements which are in term calling user-defined methods are containing business-logic.



Comments

- A comment is a non-executable statement that helps to read and understand a program especially when your programs get more complex.
- Java programming language supports three types of comments.
 - Single line (or end-of line) comment**: It starts with a double slash symbol (//) and terminates at the end of the current line. The compiler ignores everything from // to the end of the line. For example:

`// Calculate sum of two numbers`

- Multiline Comment**: Java programmer can use C/C++ comment style that begins with delimiter `/*` and ends with `*/`. All the text written between the delimiter is ignored by the compiler. This style of comments can be used on part of a line, a whole line or more commonly to define multi-line comment. For example.

`/*calculate sum of two numbers */`

- Documentation comments**: This comment style is new in Java. Such comments begin with delimiter `/**` and end with `*/`. The compiler also ignores this type of comments just like it ignores comments that use `/*` and `*/`. The main purpose of this type of comment is to automatically generate program documentation. The java doc tool reads these comments and uses them to prepare your program's documentation in HTML format. For example.

`/**The text enclosed here will be part of program documentation */`

14. Describe the control statements in JAVA?

ANSWER: Refer notes of Module 2

15. Write a Java program that accepts N integers through console and sort them in ascending order.

ANSWER:

```
import java.util.Scanner;
public class Ascending_Order
{
```

```

public static void main(String[] args)
{
    int n, temp;
    Scanner s = new Scanner(System.in);
    System.out.print("Enter no. of elements you want in array:");
    n = s.nextInt();
    int a[] = new int[n];
    System.out.println("Enter all the elements:");
    for (int i = 0; i < n; i++)
    {
        a[i] = s.nextInt();
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    System.out.print("Ascending Order:");
    for (int i = 0; i < n - 1; i++)
    {
        System.out.print(a[i] + ",");
    }
    System.out.print(a[n - 1]);
}
}

```

Output:

```

$ javac Ascending
_Order.java
$ java Ascending _Order
Enter no. of elements you
want in array:5
Enter all the elements:
4
3
2
6
1
Ascending Order:1,2,3,4,6

```

16. Explain in detail how exception handling mechanism used in Java using throw and throws?

ANSWER:

Java throw keyword

- The Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or unchecked exception in java by throw keyword.
- The syntax of java throw keyword is given below.

throw exception;

```

public class MyClass {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Access denied - You must be
at least 18 years old.");
        }
        else {
            System.out.println("Access granted - You are old enough!");
        }
    }

    public static void main(String[] args) {
        checkAge(15); // Set age to 15 (which is below 18...)
    }
}

```

OUTPUT

```

Exception in thread "main"
java.lang.ArithmeticException: Access denied -
You must be at least 18 years old.

        at MyClass.checkAge(MyClass.java:4)

        at MyClass.main(MyClass.java:12)

```

Java throws keyword

- The Java throws keyword is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Syntax of java throws

```
return_type method_name() throws exception_class_name  
{  
    //method code  
}
```

```
import java.io.*;  
class M{  
    void method()throws IOException{  
        throw new IOException("device error");  
    }  
}  
public class Testthrows4{  
    public static void main(String args[])throws IOException{//declare exception  
        M m=new M();  
        m.method();  
  
        System.out.println("normal flow...");  
    } }
```

OUTPUT

```
Exception in thread "main" java.io.IOException: device  
error at M.method(Testthrows4.java:4)  
        at Testthrows4.main(Testthrows4.java:10)
```

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.



17. **What is Thread Synchronization? With an example illustrate the working of any one technique used for Thread synchronization in Java?**

OR

What is the use of synchronized keyword in java?

THREAD SYNCHRONIZATION

- When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally, they can produce unforeseen result due to concurrency issues.
- For example, if multiple threads try to write within a same file then they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.
- So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time.
- Following is the general form of the synchronized statement :
 - Syntax

```
synchronized (object identifier)
{ // Access shared variables and other shared resources
}
```

Understanding the problem without Synchronization

- In this example, we are not using synchronization and creating multiple threads that are accessing printTable() method and produce the random output.

```
class Table{
    void printTable(int n){//method not synchronized
        for(int i=1;i<=5;i++){
            System.out.println(n*i);
        }
    }
}
```

```
class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread{
    Table t;
    MyThread2(Table t){
```

OUTPUT

5
10
100
200
300
400
500
15
20
25


```

this.t=t; }

public void run(){
t.printTable(100);
} }

class TestSynchronization1 {
public static void main(String args[]){
Table obj = new Table();//only one object
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start(); t2.start(); }}

```

Synchronized Keyword

- To synchronize above program, we must synchronize access to the shared printTable () method, making it available to only one thread at a time. This is done by using keyword synchronized with printTable () method.
- With a synchronized method, the lock is obtained for the duration of the entire method.
- So if you want to lock the whole object, use a synchronized method

```

synchronized void printTable(int n) { }

```

- Example : Implementation of synchronized method

```

class Table{

    synchronized void printTable(int n){//method not synchronized
    for(int i=1;i<=5;i++){
        System.out.println(n*i);
    } }}

class MyThread1 extends Thread{
Table t;
MyThread1(Table t){
this.t=t;
}
public void run(){
t.printTable(5);
} }

class MyThread2 extends Thread{
Table t;

```

```

MyThread2(Table t){
this.t=t;
}
public void run(){
t.printTable(100);
} }
class TestSynchronization1 {
public static void main(String args[]){
Table obj = new Table();//only one object
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
} }

```

OUTPUT

```

5
10
15
20
25
100
200
300
400
500

```

18. Discuss any five methods used for string processing in Java with examples?

ANSWER

1. **STRING LENGTH (length())**

- The java string length () method gives length of the string.
- It returns count of total number of characters.
- Syntax: length();

Java program illustrating the use of String Length : length()

```

class Main
{
public static void main(String args[])
{
String S1 = "JAVA IS SIMPLE";
String S2 = "JAVA IS POWERFUL";
int len1 = S1.length(); // returns the no of characters in the string S1
int len2 = S2.length();
System.out.println("Length of S1 is :" +len1); //14
System.out.println("Length of S2 is :" +len2); //16
}}

```

OUTPUT

```

Length of S1 is :14
Length of S2 is :16

```

2. **Java string equals()**

- method compares the two given strings based on the content of the string. If any character is not matched, it returns false. If all characters are matched, it returns true.

- Syntax:

```

▪ S1.equals(S2)

```

- Returns true if S1=S2

Java String equalsIgnoreCase()

- method compares the two given strings based on content of the string irrespective of case of the string.
- It is like equals() method but doesn't check case. If any character is not matched, it returns false otherwise it returns true.
- Syntax:

S1.equalsIgnoreCase(S2)

Returns true if S1=S2 ignoring the case of characters

class Main

```
{
    public static void main(String args[])
    {
        String S1 = "HAI";
        String S2 = "HAI";
        String S3 = new String("Hai");
        System.out.println(S1.equals(S2)); //Returns true
        System.out.println(S1.equals(S3)); //Returns false
        System.out.println(S1.equalsIgnoreCase(S2)); //Returns true
        System.out.println(S1.equalsIgnoreCase(S3)); // Returns true
    }
}
```

OUTPUT

true

false

true

true

3. Java String compareTo()

- The java string compareTo() method compares the given string with current string lexicographically.
- Syntax:

S1.compareTo(S2)

if s1 > s2, it returns positive number

if s1 < s2, it returns negative number

if s1 == s2, it returns 0

class Main

```
{
    public static void main(String args[])
    {
        String s1="hello";
        String s2="hello";
        String s3="mello";
        String s4="flag";
        System.out.println(s1.compareTo(s2));
        System.out.println(s1.compareTo(s3));
        System.out.println(s1.compareTo(s4));
    }
}
```

OUTPUT

0

-5

2

4. Java String toLowerCase ()

- method returns the string in lowercase letter. In other words, it converts all the characters of the string into lower case letter.

- Syntax:

- S1.toLowerCase()

- Converts the String S1 to lowercase

5. Java String toUpperCase ()

- method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.
- Syntax:
 - S1.toUpperCase()
 - Converts the String S1 to uppercase

```
class Main
{
    public static void main (String args [])
    {
        String S1="hello string";
        String S2="HAI JAVA";
        String s1upper=S1.toUpperCase();
        System.out.println(s1upper);
        String S2lower=S2.toLowerCase();
        System.out.println(S2lower);
    }
}
```

OUTPUT

HELLO STRING

hai java

19. Write a Java program to implement a basic calculator.

ANSWER

Refer Lab Record

20. Explain the steps using for connecting a Java program to a database using JDBC API with a proper example.

ANSWER

Refer note of Module 5

